RS Latch

Theory. Cross-coupled NOR gates provide feedback storage. The upper output is Q and the lower is Qn. With active-high inputs, S sets Q and R resets Q. At S=R=0 the settled state is retained. The equations are Q = NOT(R OR Qn) and Qn = NOT(S OR Q).

S	R	Q(next)	Qn(next)	Function
0	0	Q(previous)	Qn(previous)	Hold
1	0	1	0	Set
0	1	0	1	Reset
1	1	0	0	Forbidden

Verification procedure. Initialize with S=1, R=0; release S and check retention. Assert R with S=0 and release R to check retention of zero. Finally assert S=R=1 and observe both outputs low. Avoid treating simultaneous release of the forbidden state as a deterministic operation.

Result and scope. The five screenshot pairs show hold at Q=1, set, reset, hold at Q=0, and the forbidden input. Both implementations display the expected static values. The forbidden state is not complementary; its release behavior and physical metastability are not established by these images.

VHDL Source | RS_Latch

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RS_Latch is
    Port (
        S  : in  STD_LOGIC;
        R  : in  STD_LOGIC;
        Q  : out STD_LOGIC;
        Qn : out STD_LOGIC
    );
end RS_Latch;

architecture Behavioral of RS_Latch is
    signal q_state : STD_LOGIC;
begin
    process(S, R)
    begin
        if R = '1' then
            q_state <= '0';
        elsif S = '1' then
            q_state <= '1';
        end if;
    end process;

    Q  <= q_state;
    Qn <= '0' when (S = '1' and R = '1')
          else not q_state;
end Behavioral;
```

R has priority in q_state; Qn is explicitly forced to 0 for S=R=1. This reproduces the displayed forbidden-state outputs, but deterministic VHDL release behavior is not a model of physical metastability.

RS Latch | Circuit Diagram

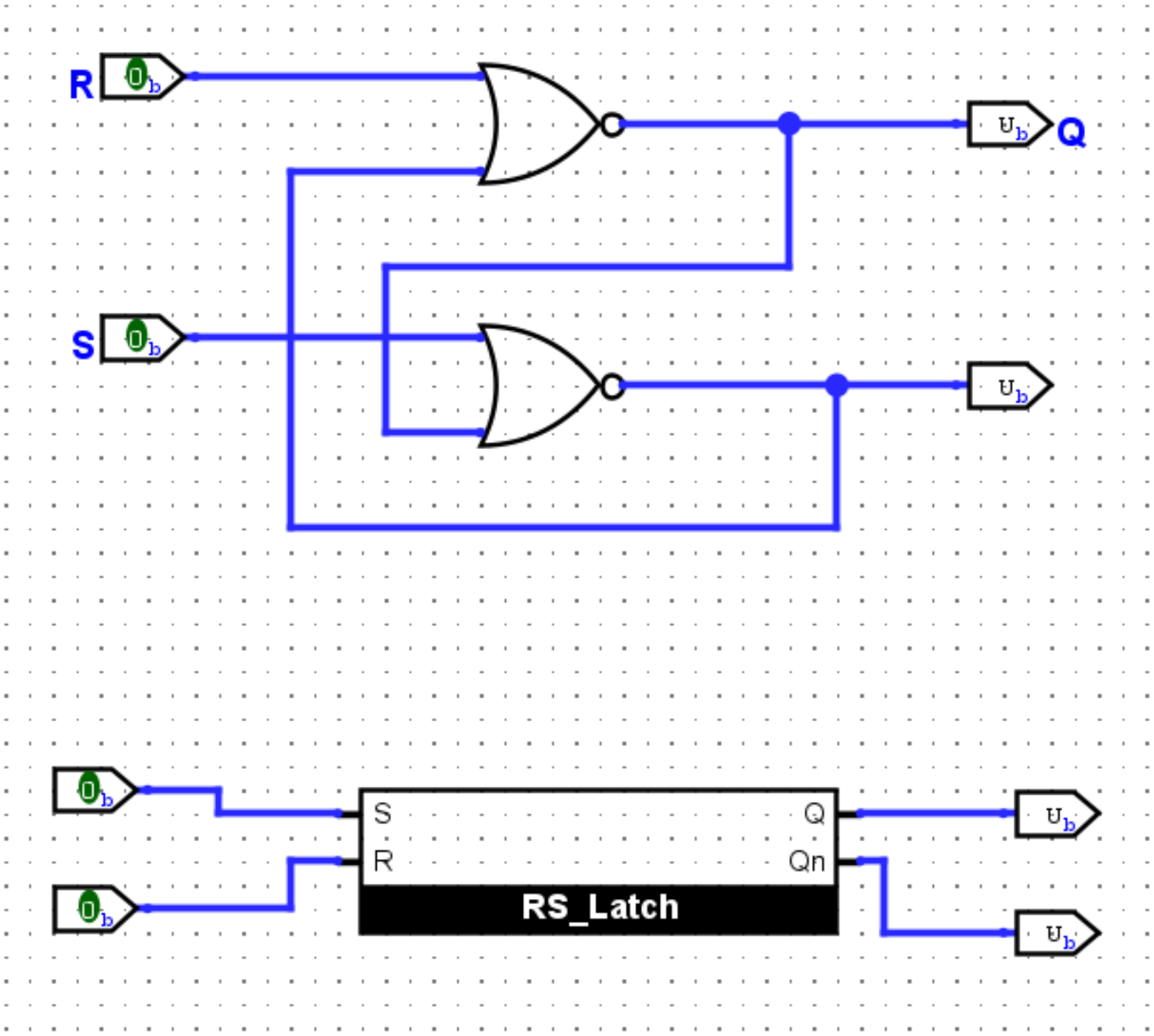
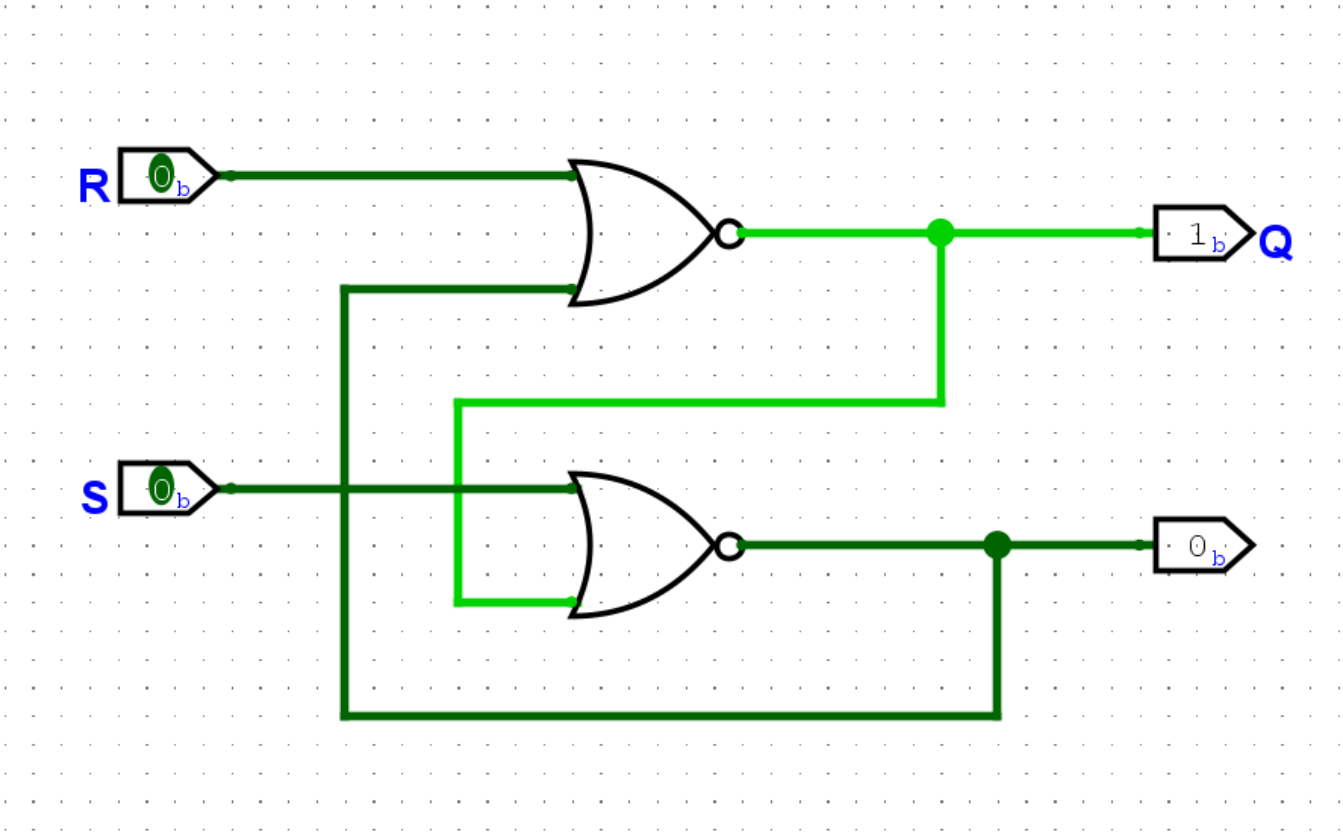


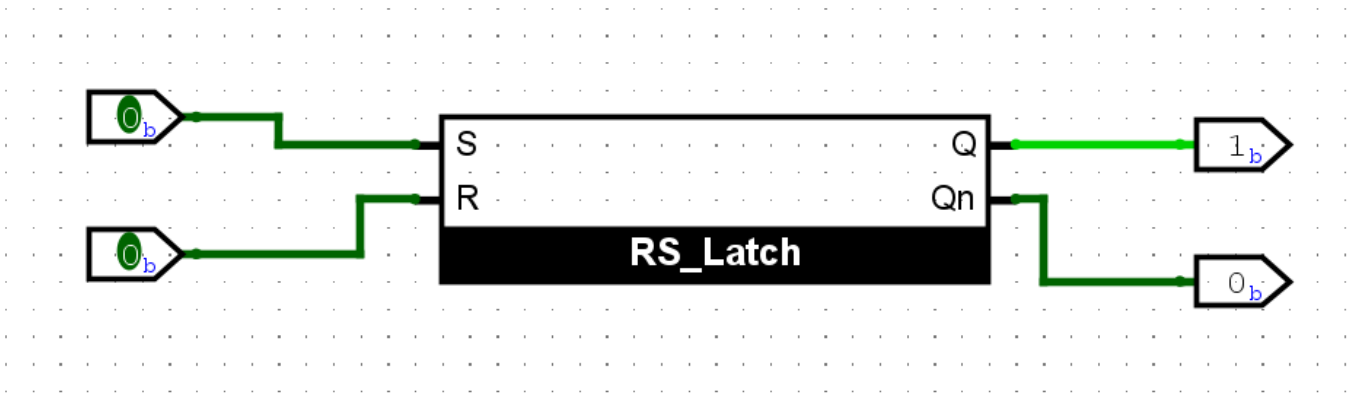
Figure 01.D. Circuit diagram. RS Latch.

RS Latch | Test 1

Gate-level circuit



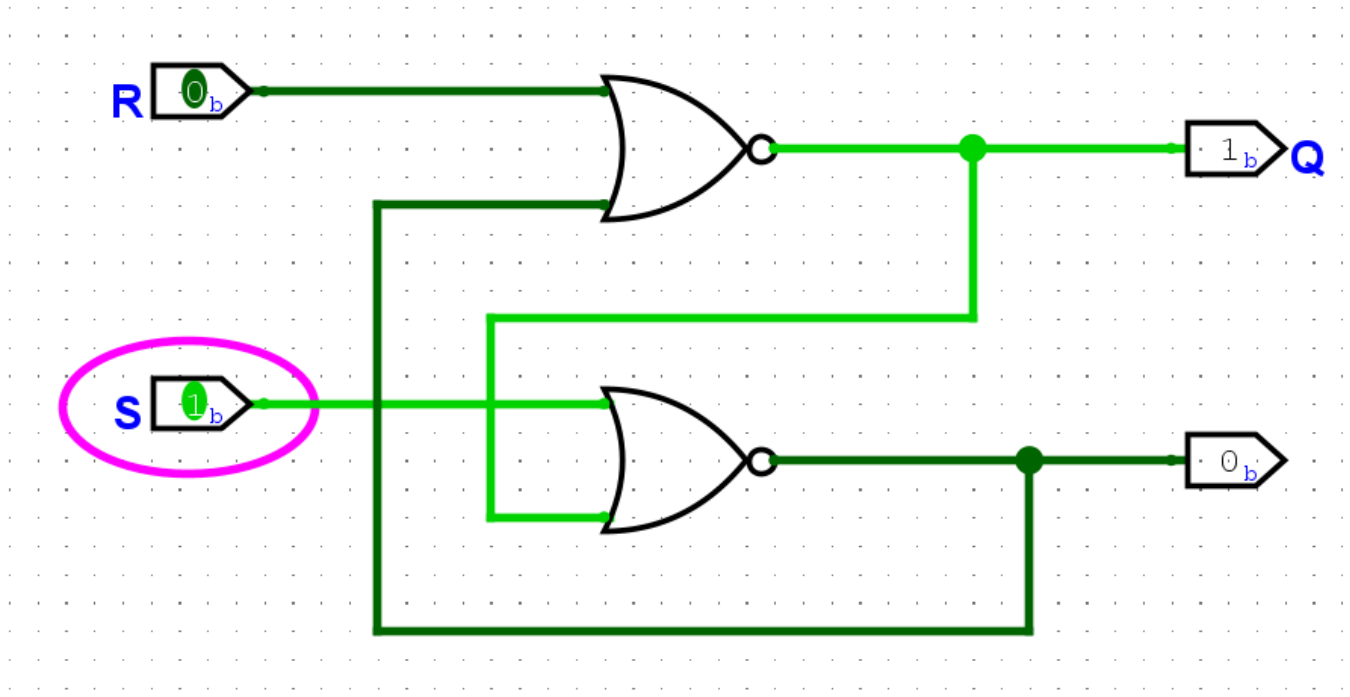
VHDL component



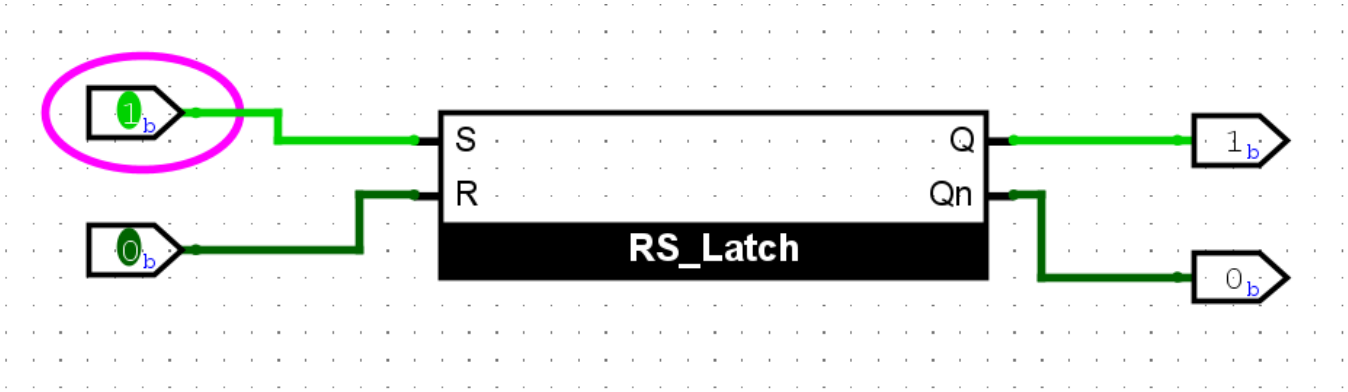
Hold: S=R=0; Q=1, Qn=0. Stored one is visible.

RS Latch | Test 2

Gate-level circuit



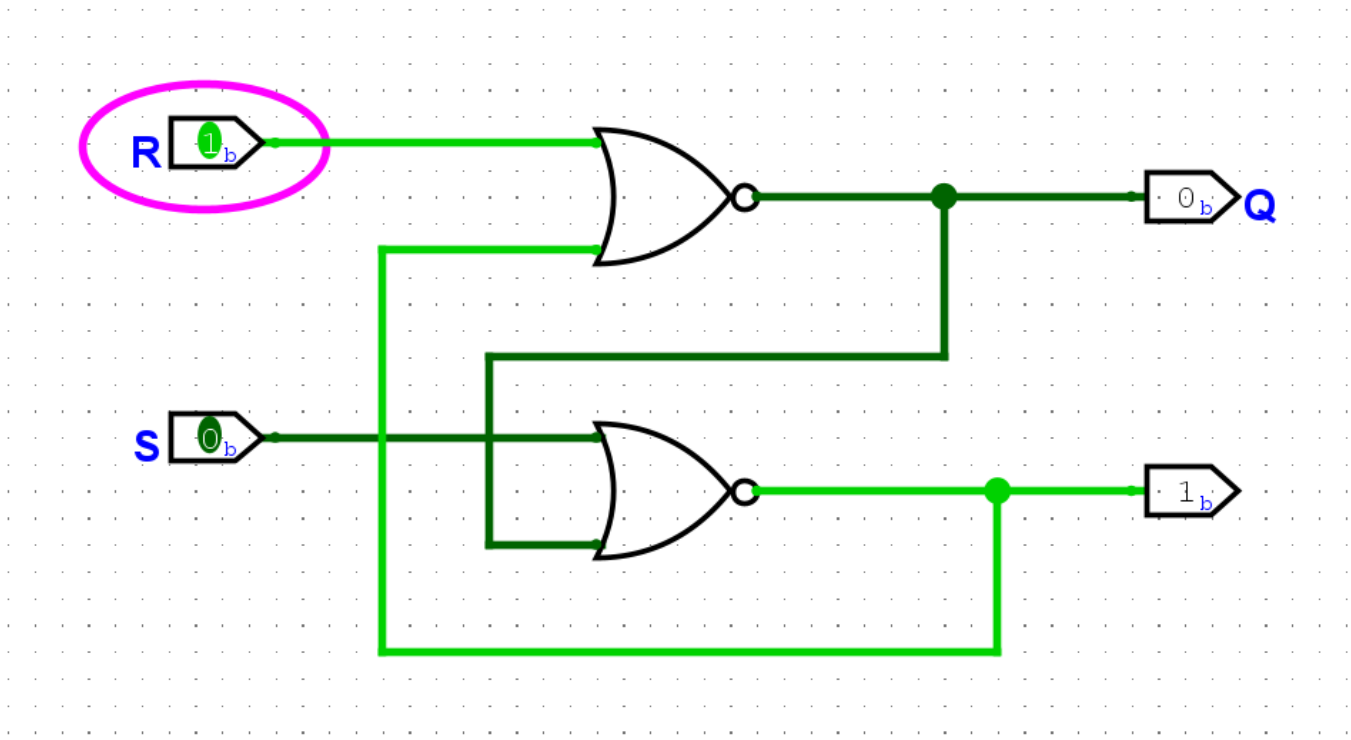
VHDL component



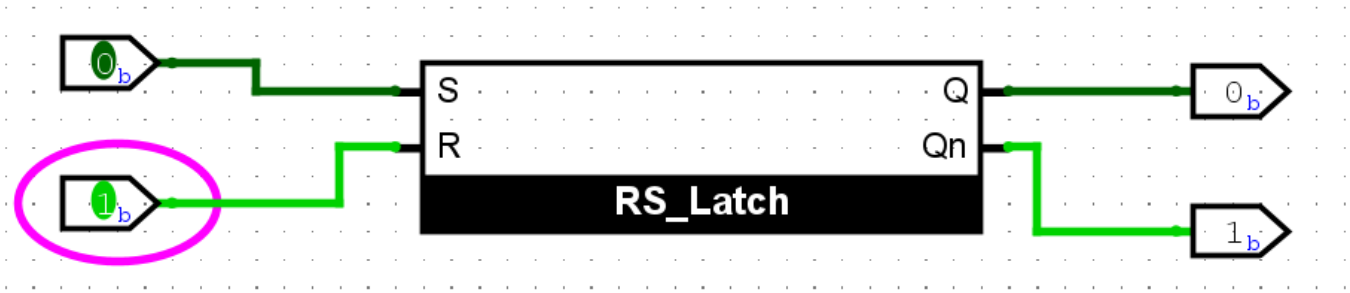
Set: S=1, R=0; Q=1, Qn=0.

RS Latch | Test 3

Gate-level circuit



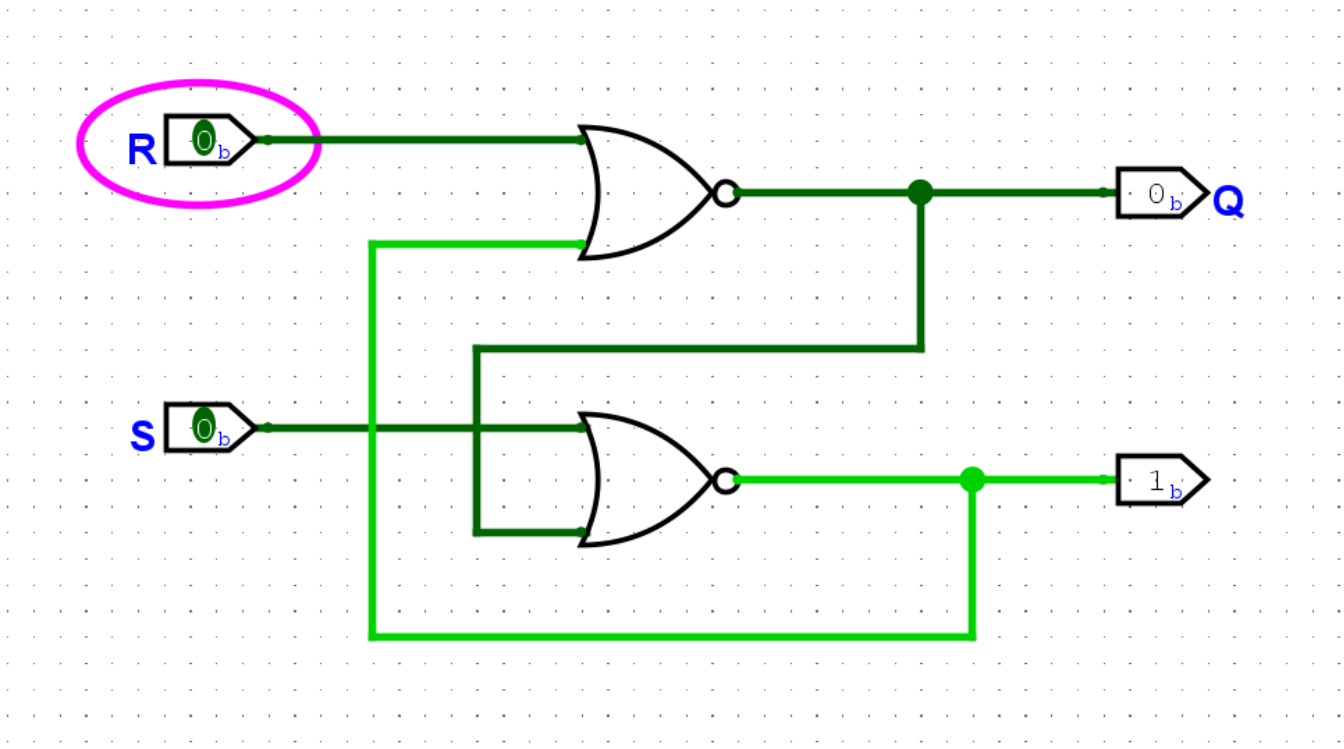
VHDL component



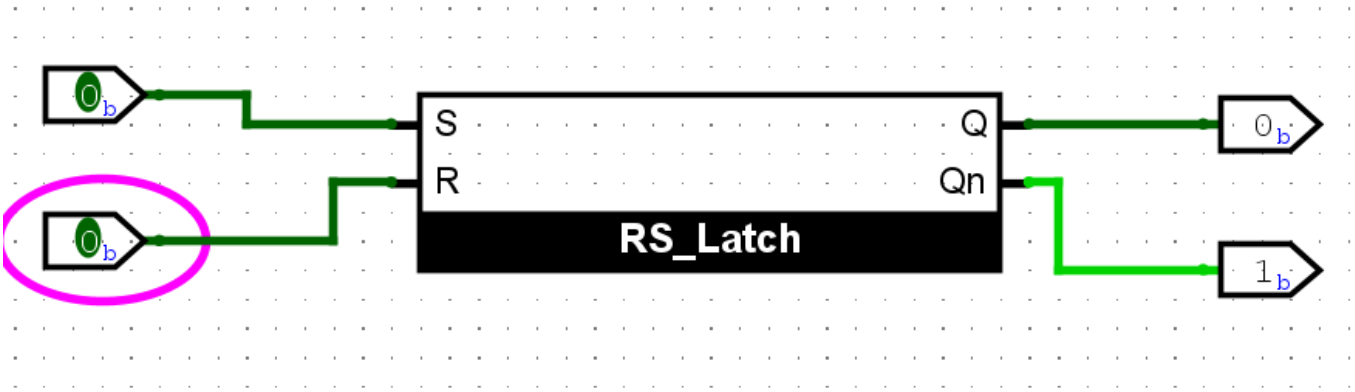
Reset: S=0, R=1; Q=0, Qn=1.

RS Latch | Test 4

Gate-level circuit



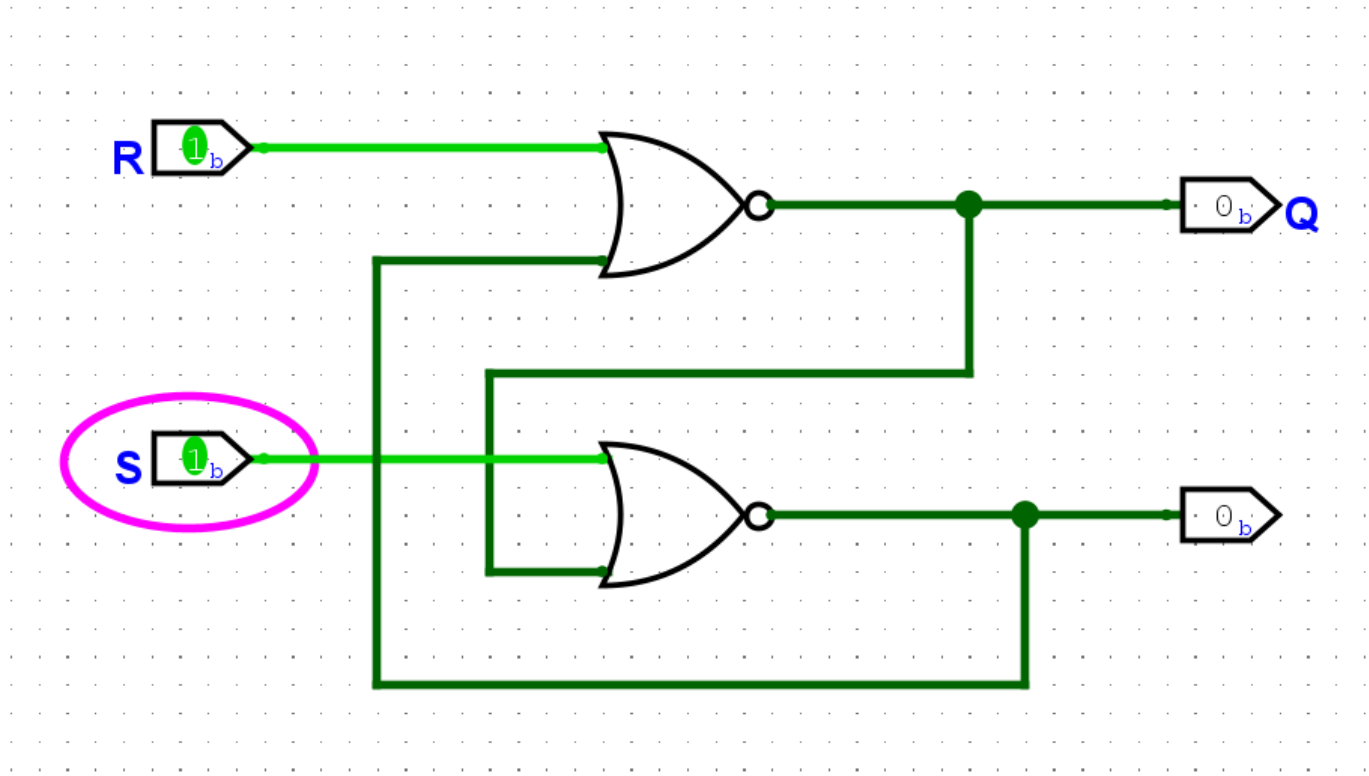
VHDL component



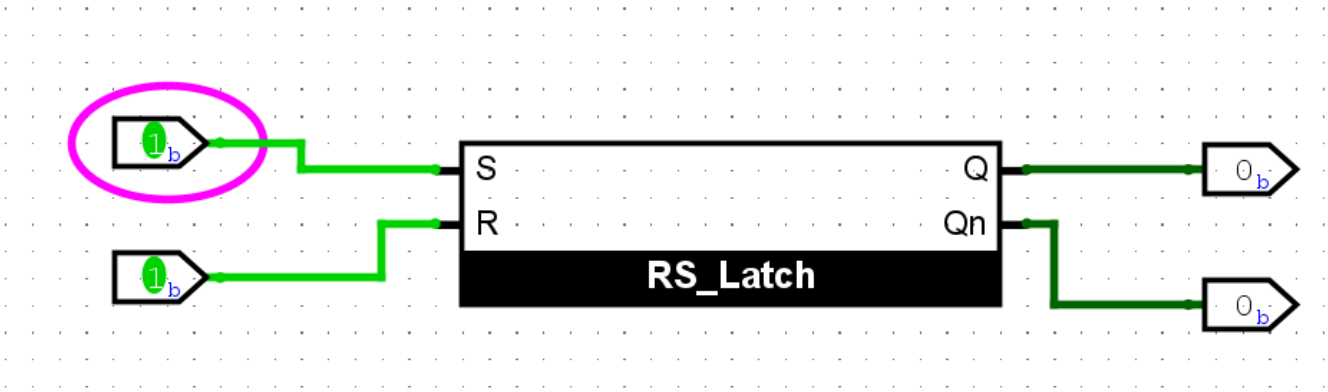
Hold: S=R=0; Q=0, Qn=1. Stored zero is visible.

RS Latch | Test 5

Gate-level circuit



VHDL component



Forbidden: S=R=1; Q=0, Qn=0. Both outputs are forced low.

D Latch

Theory. The D latch uses input gating and feedback to avoid the normal RS forbidden input combination. It is level-sensitive: while en=1, Q follows D after propagation delay. When en=0, the feedback path preserves the previously stored value. Q(next) = en D OR (NOT en) Q(previous).

en	D	Q(next)	Function
0	X	Q(previous)	Hold
1	0	0	Transparent: store 0
1	1	1	Transparent: store 1

Verification procedure. With en=1, change D from 0 to 1 and confirm Q follows. Store zero and disable en; vary D and check retention. Independently store one, disable en, and repeat. Re-enable with D=0 and confirm Q returns to zero.

Result and scope. Images 1 and 2 demonstrate transparency with en=1. Hold-one is verified by Figures 02.4 and 02.6: with en=0, changing D from 1 to 0 leaves Q=1 and Qn=0. Hold-zero is verified by Figures 02.3 and 02.7: with en=0, changing D from 0 to 1 leaves Q=0 and Qn=1. These are two separately initialized hold tests. Both the gate circuit and VHDL implementation retain the stored value while disabled.

VHDL Source | D_Latch

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_Latch is
    Port (
        D   : in  STD_LOGIC;
        en  : in  STD_LOGIC;
        Q   : out STD_LOGIC;
        Qn  : out STD_LOGIC
    );
end D_Latch;

architecture Behavioral of D_Latch is
    signal q_state : STD_LOGIC;
begin

    process(D, en)
    begin
        if en = '1' then
            q_state <= D;
        end if;
    end process;

    Q  <= q_state;
    Qn <= not q_state;

end Behavioral;
```

process(D, en) updates q_state only when en=1. With no assignment when disabled, the stored value is retained, implementing an active-high D latch.

D Latch | Circuit Diagram

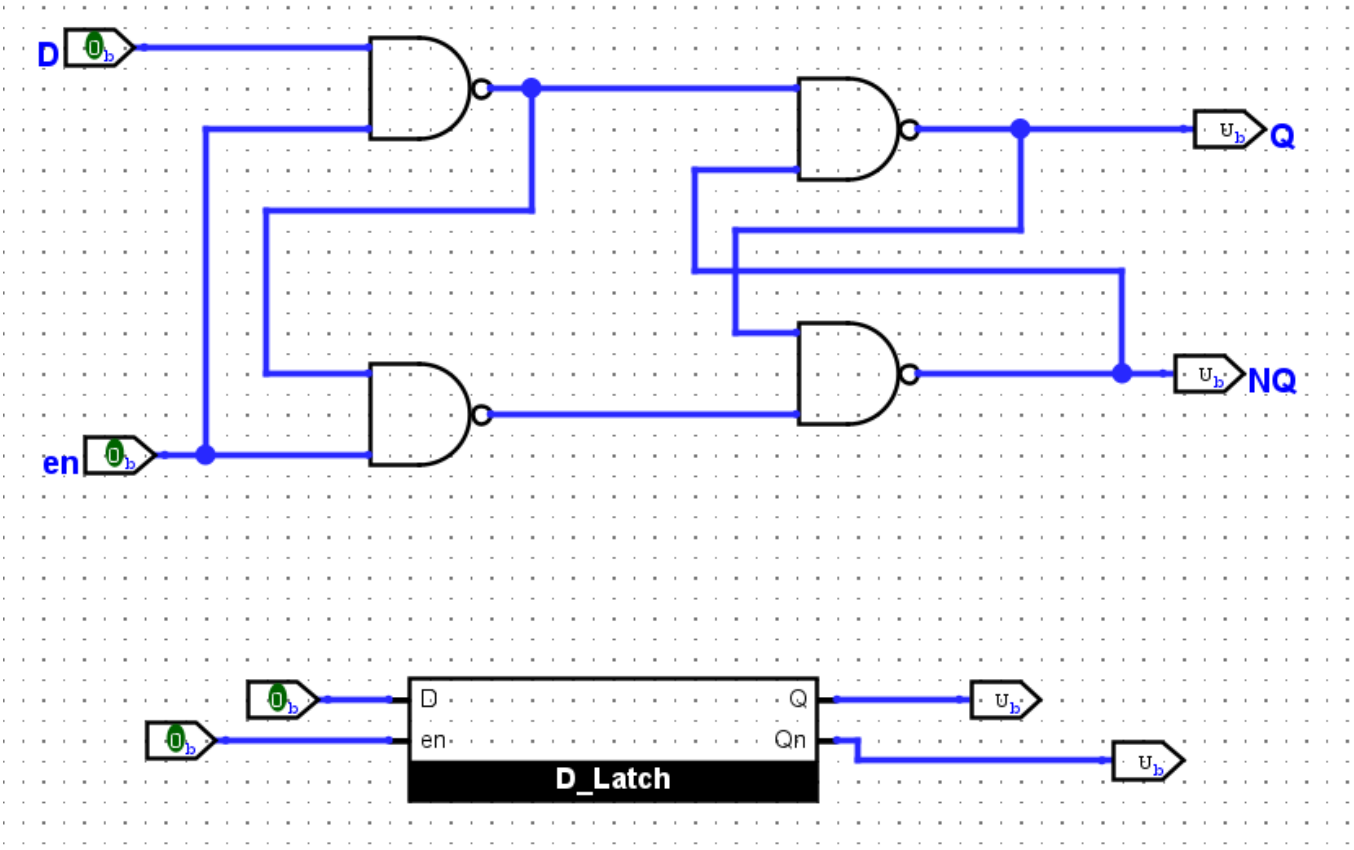


Figure 02.D. Circuit diagram. D Latch.

D Latch | Screenshots

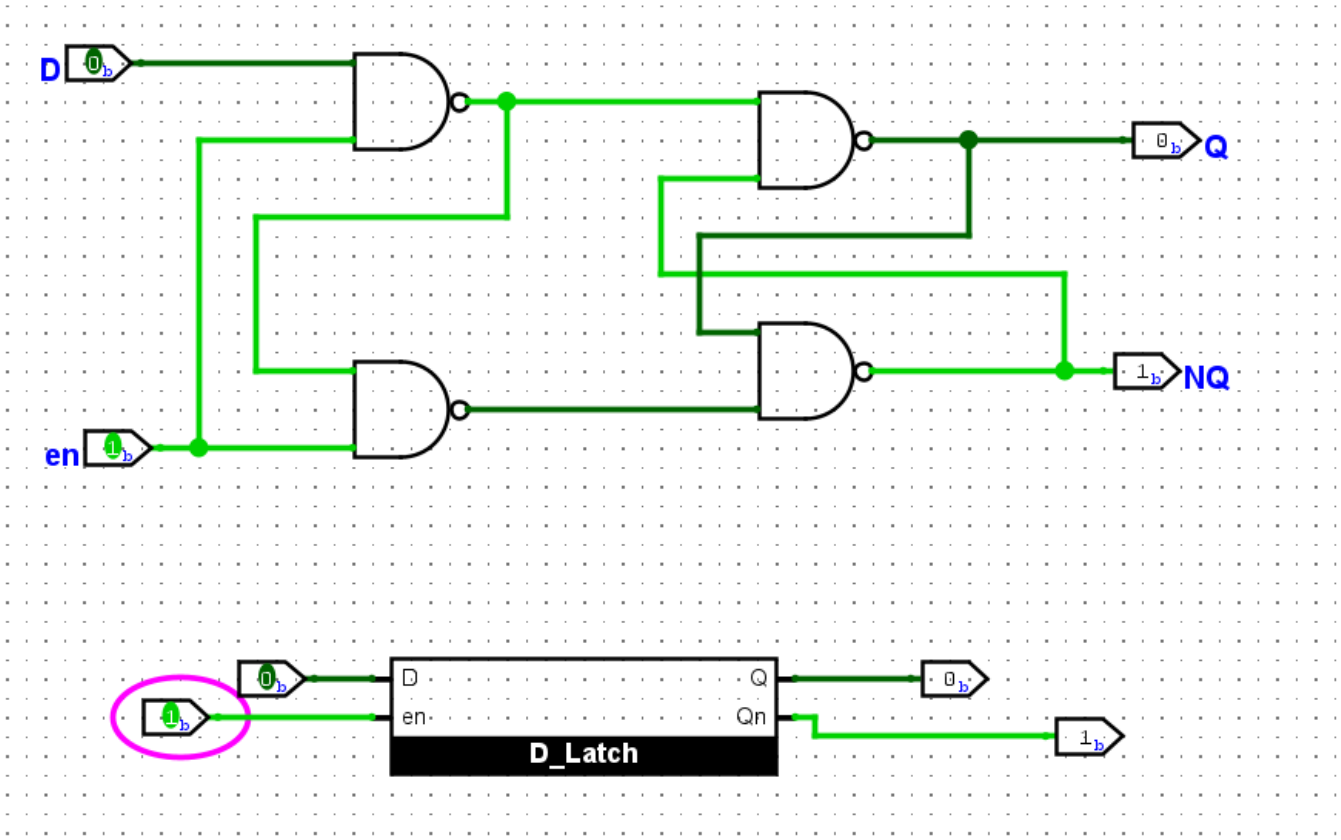


Figure 02.1. en=1, D=0: both circuits show Q=0, Qn=1.

D Latch | Screenshots

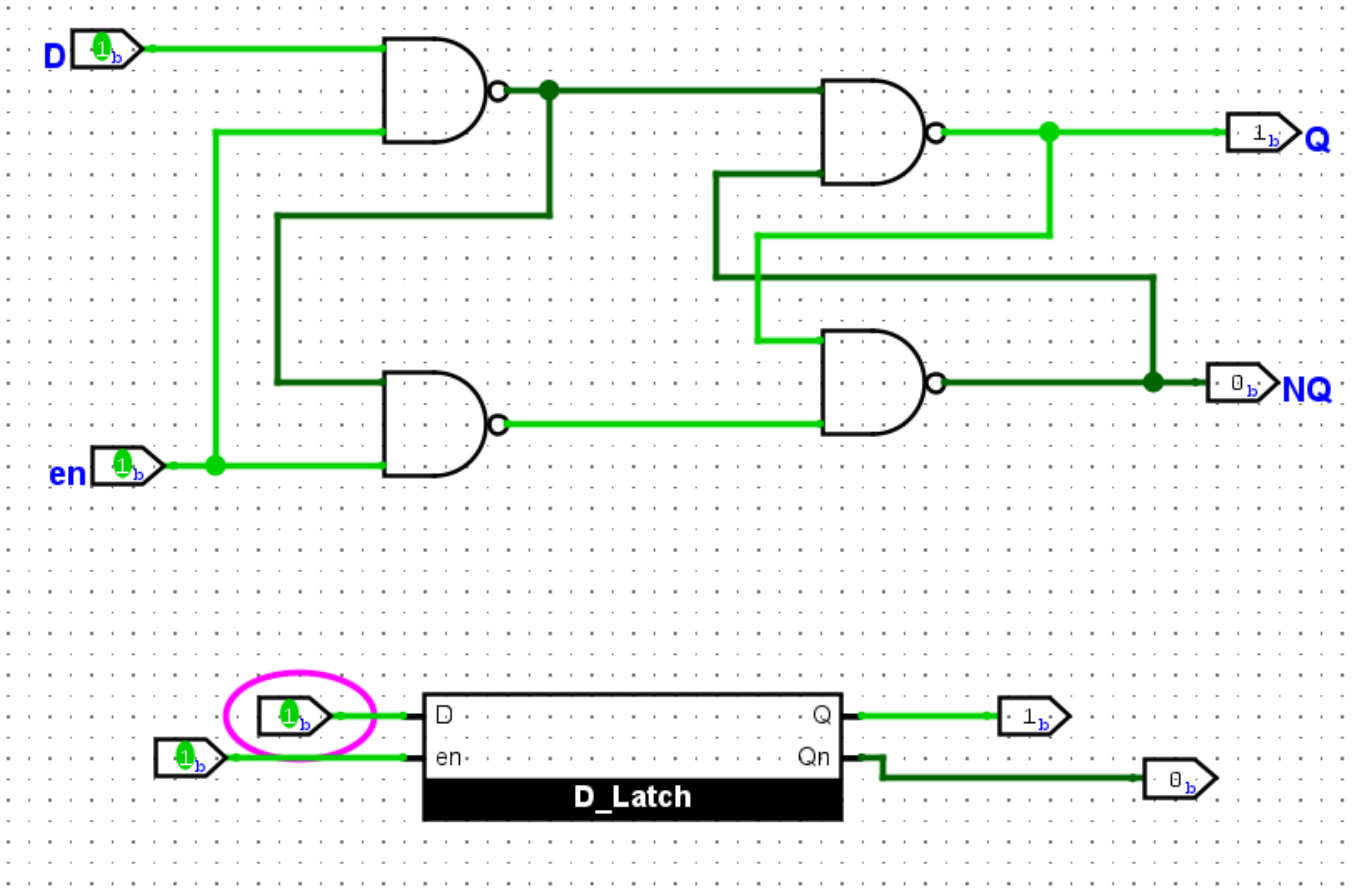


Figure 02.2. en=1, D=1: both show Q=1, Qn=0.

D Latch | Screenshots

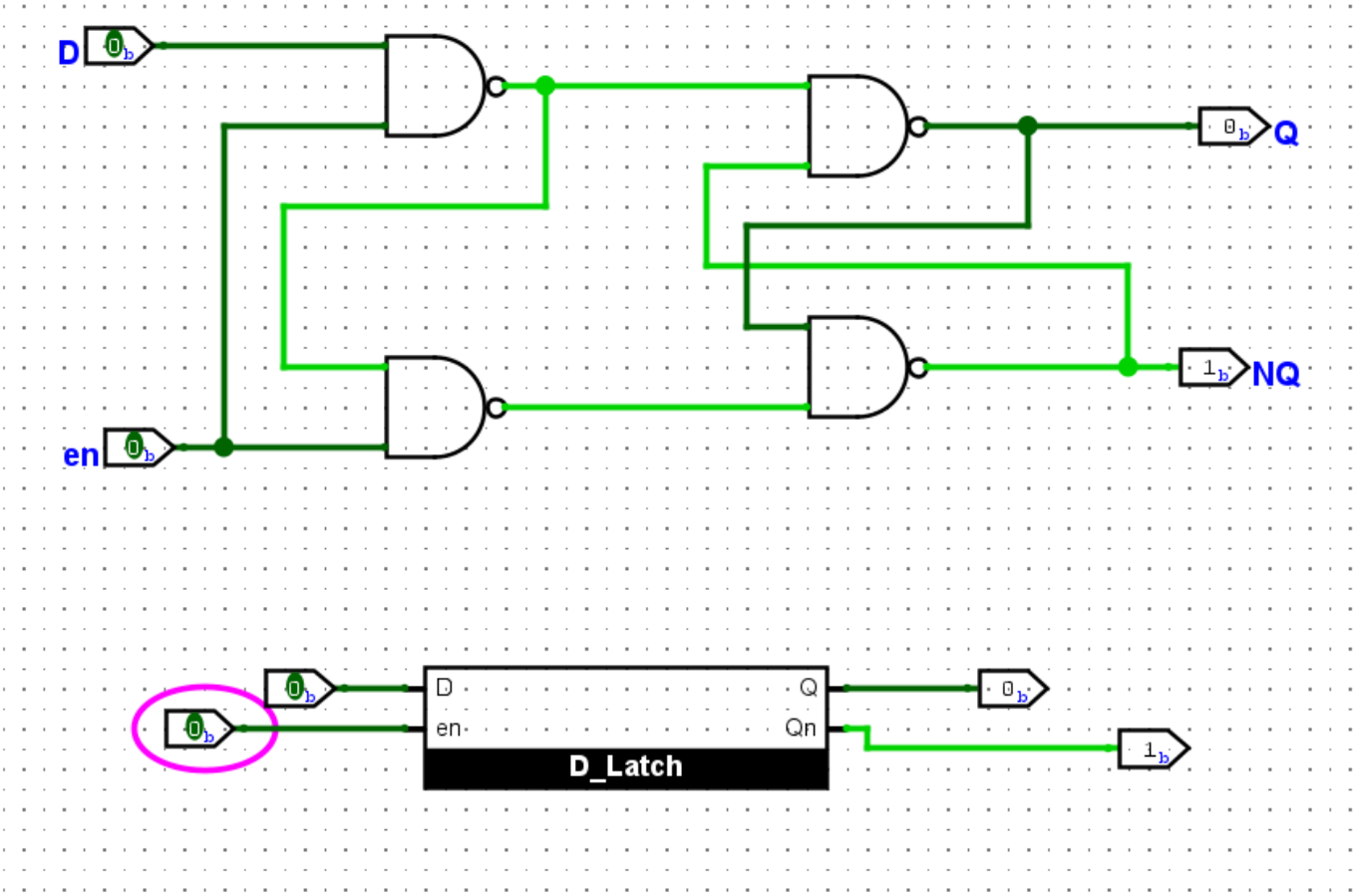


Figure 02.3. en=0, D=0: both show the stored zero.

D Latch | Screenshots

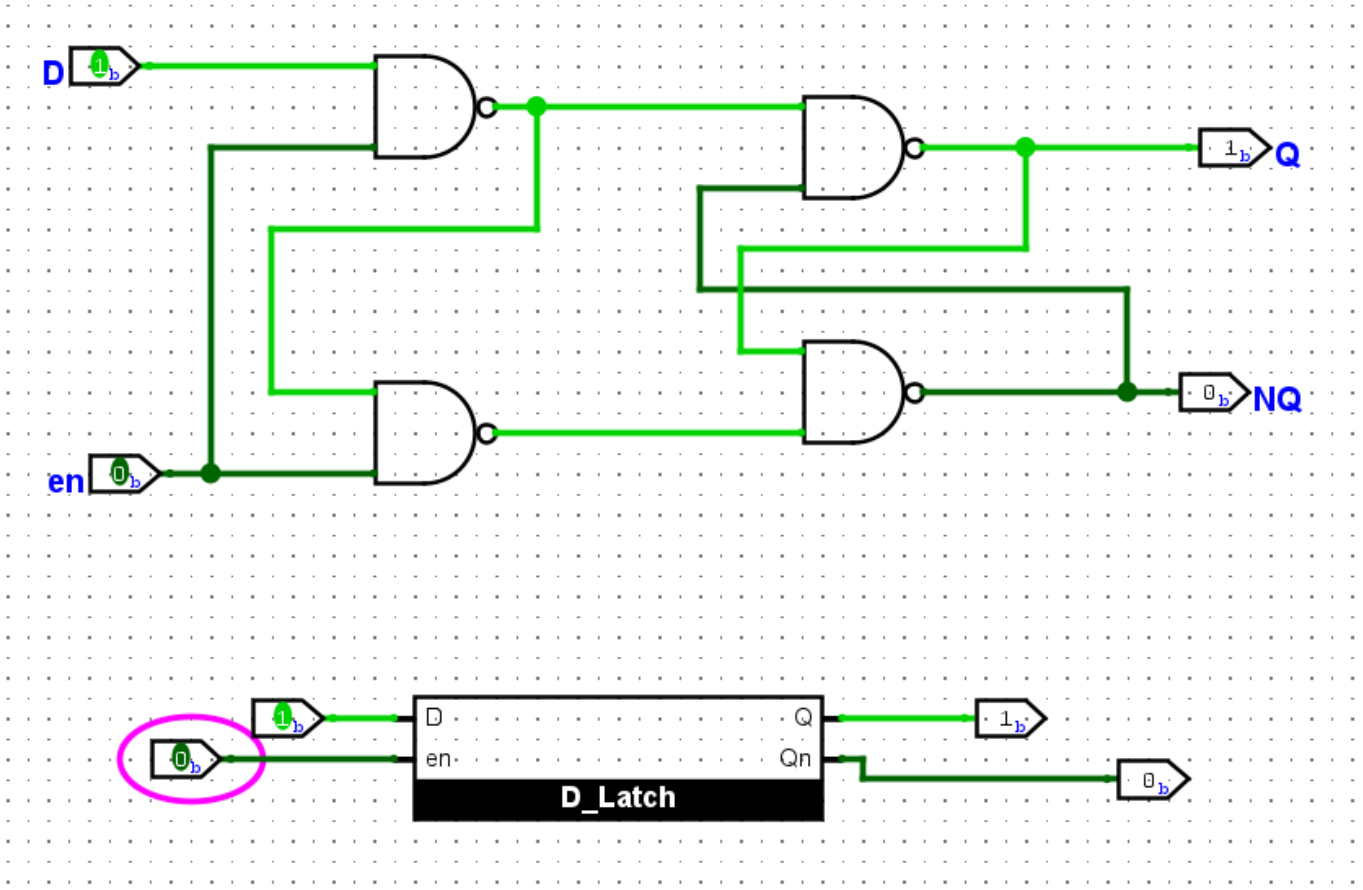


Figure 02.4. en=0, D=1: both show a stored one; this is an independent hold configuration.

D Latch | Screenshots

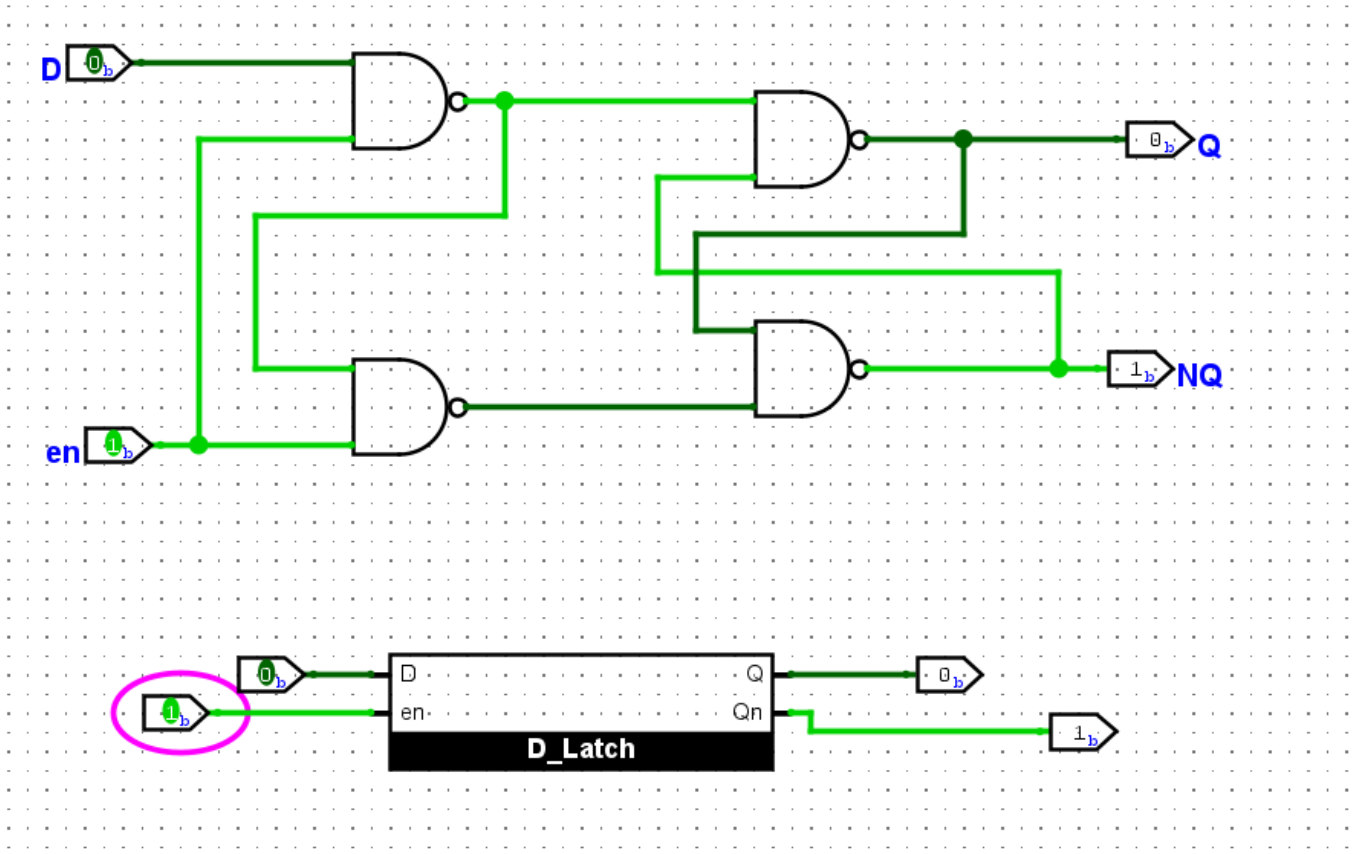


Figure 02.5. en=1, D=0: both return to Q=0, Qn=1.

D Latch | Hold Verification

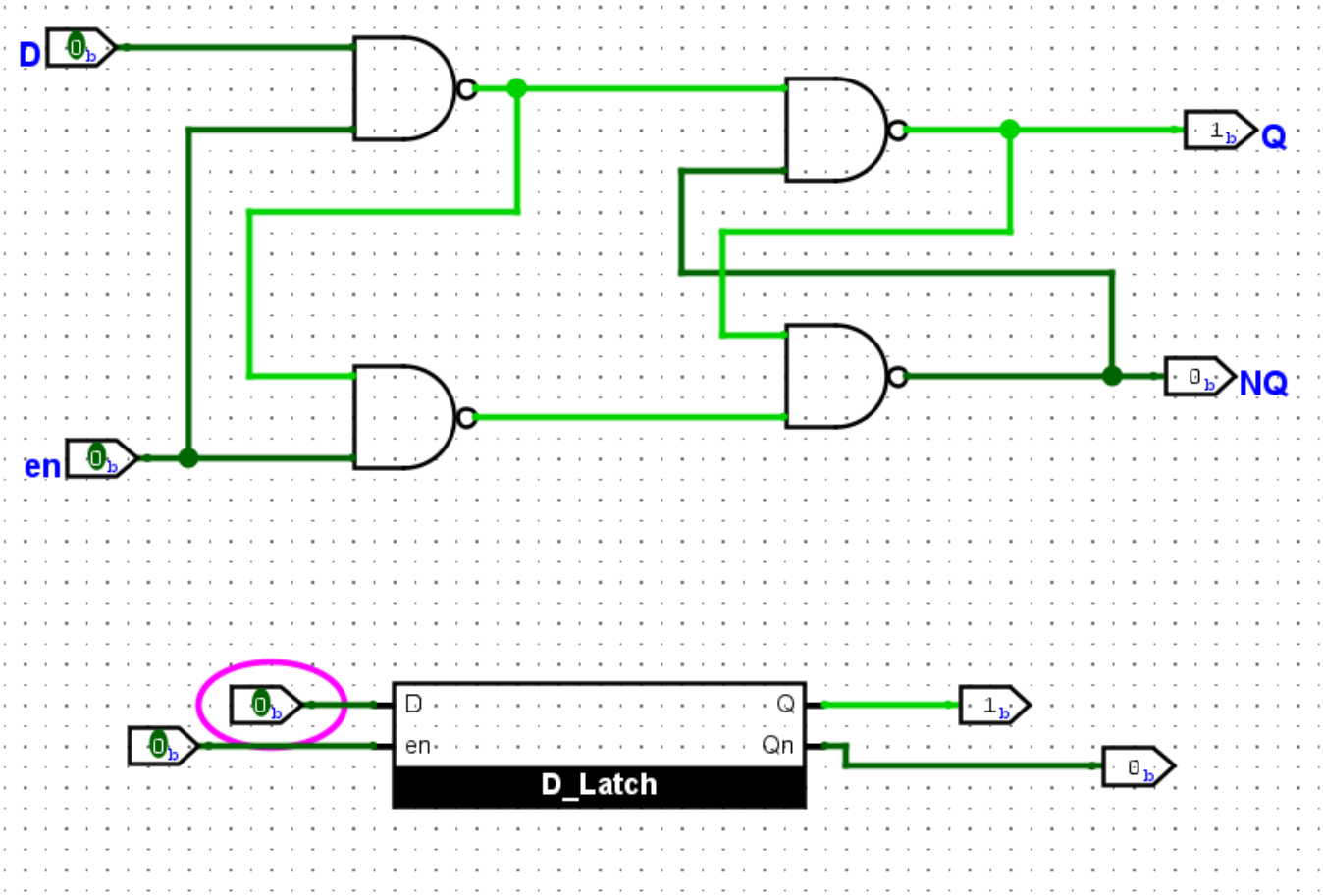


Figure 02.6. en=0, D=0: both circuits retain Q=1, Qn=0. Compared with Figure 02.4 (D=1, en=0, Q=1), changing D to 0 does not alter the stored one.

D Latch | Hold Verification

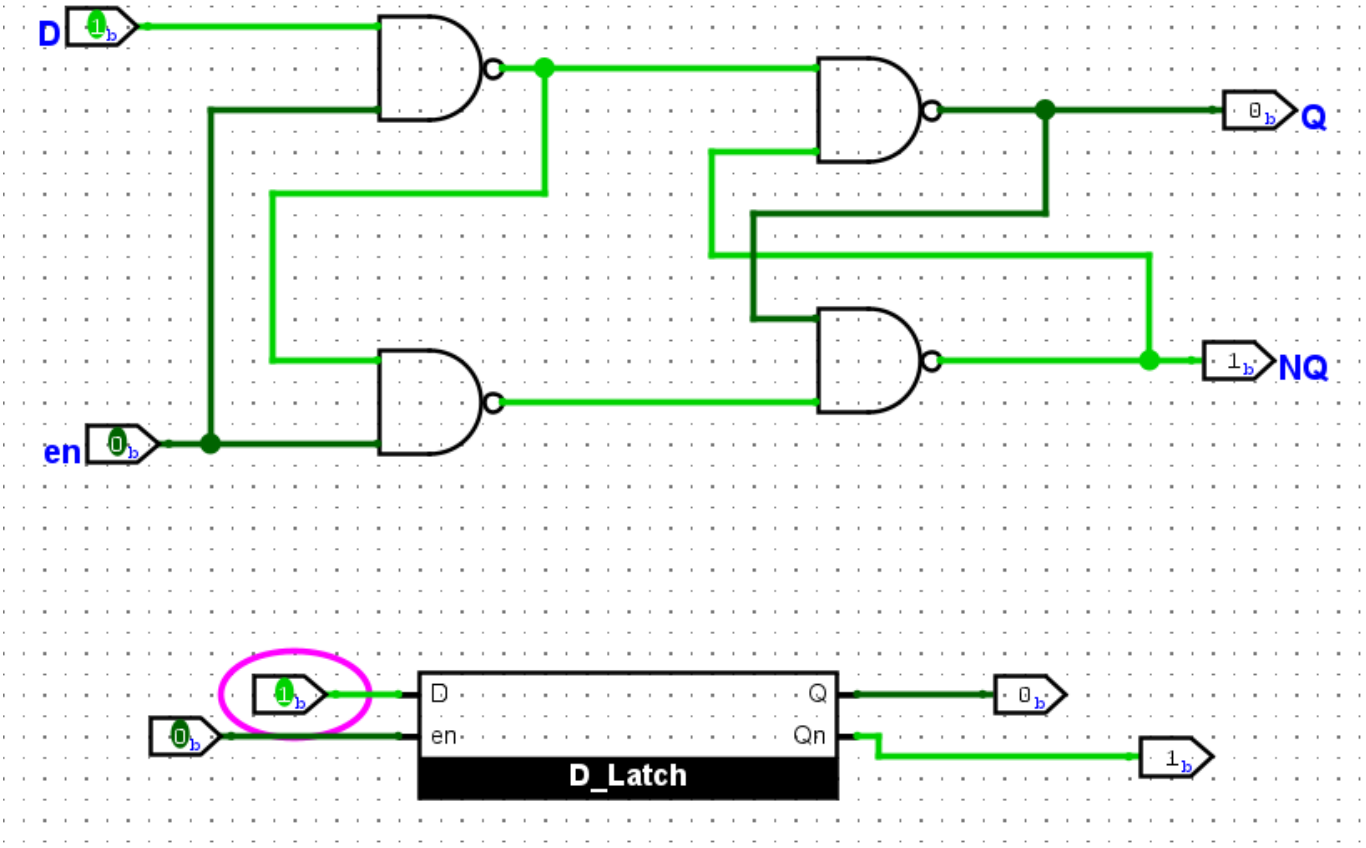


Figure 02.7. en=0, D=1: both circuits retain Q=0, Qn=1. Compared with Figure 02.3 (D=0, en=0, Q=0), changing D to 1 does not alter the stored zero.

Positive-edge D Flip-Flop

Theory. Two latches with opposite clock phases form a master-slave storage element. Here the master accepts data during the low phase and the slave transfers the stored value during the high phase, giving positive-edge behavior. In the ideal edge-triggered model, Q(next)=D only on a 0-to-1 clock transition; otherwise Q holds. The gate implementation uses eight NAND gates and one inverter. The inverter output branches to both master input gates; the slave receives CLK directly.

Clock event	D	Q(next)	Function
Rising edge	0	0	Capture 0
Rising edge	1	1	Capture 1
No rising edge	X	Q(previous)	Hold

Verification procedure. Keep CLK low and change D; check that only the master state can change. Raise CLK and check Q captures D. While CLK remains high, change D and verify Q holds. Lower CLK and then raise it again to capture the new D.

Result and scope. The single-inverter circuit and VHDL implementation show matching outputs in all five test images. With CLK=1, changing D from 0 to 1 leaves Q=0. Lowering CLK prepares the master while Q remains 0; raising CLK then stores 1 at Q. The final test stores D=0 on a subsequent rising edge. The sequence demonstrates data capture on rising edges and retention between them.

VHDL Source | D_FlipFlop

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FlipFlop is
    Port (
        D   : in  STD_LOGIC;
        CLK : in  STD_LOGIC;
        Q   : out STD_LOGIC;
        Qn  : out STD_LOGIC
    );
end D_FlipFlop;

architecture Behavioral of D_FlipFlop is
    signal q_state : STD_LOGIC;
begin

    process(CLK)
    begin
        if rising_edge(CLK) then
            q_state <= D;
        end if;
    end process;

    Q  <= q_state;
    Qn <= not q_state;

end Behavioral;
```

process(CLK) assigns D only inside rising_edge(CLK), implementing positive-edge capture. The code does not assign an initial state.

Positive-edge D Flip-Flop | Circuit Diagram

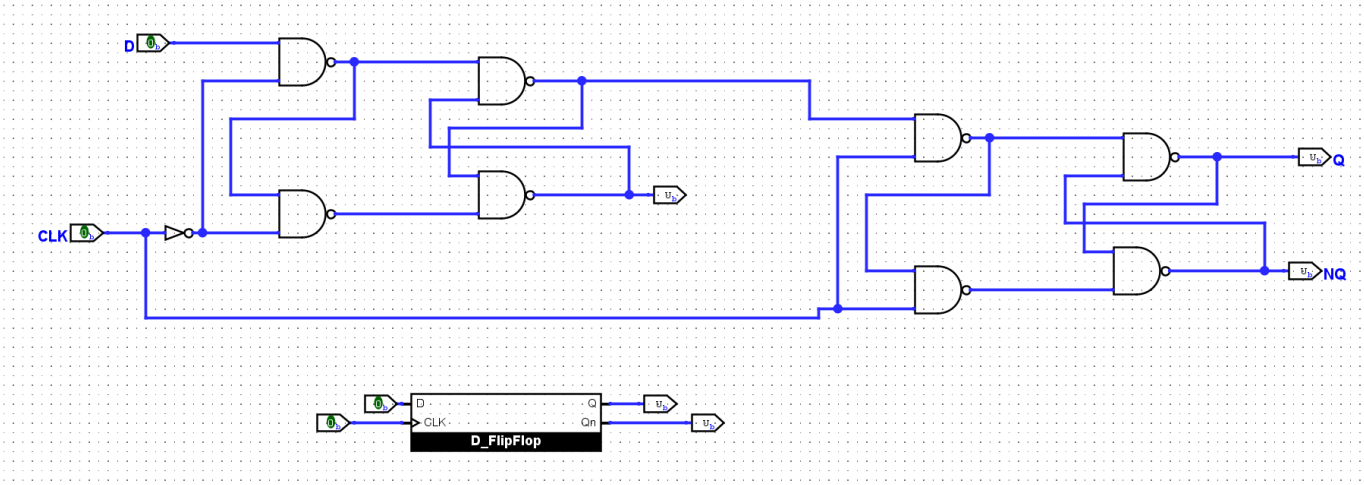


Figure 03.D. Circuit diagram. Positive-edge D Flip-Flop.

Positive-edge D Flip-Flop | Screenshots

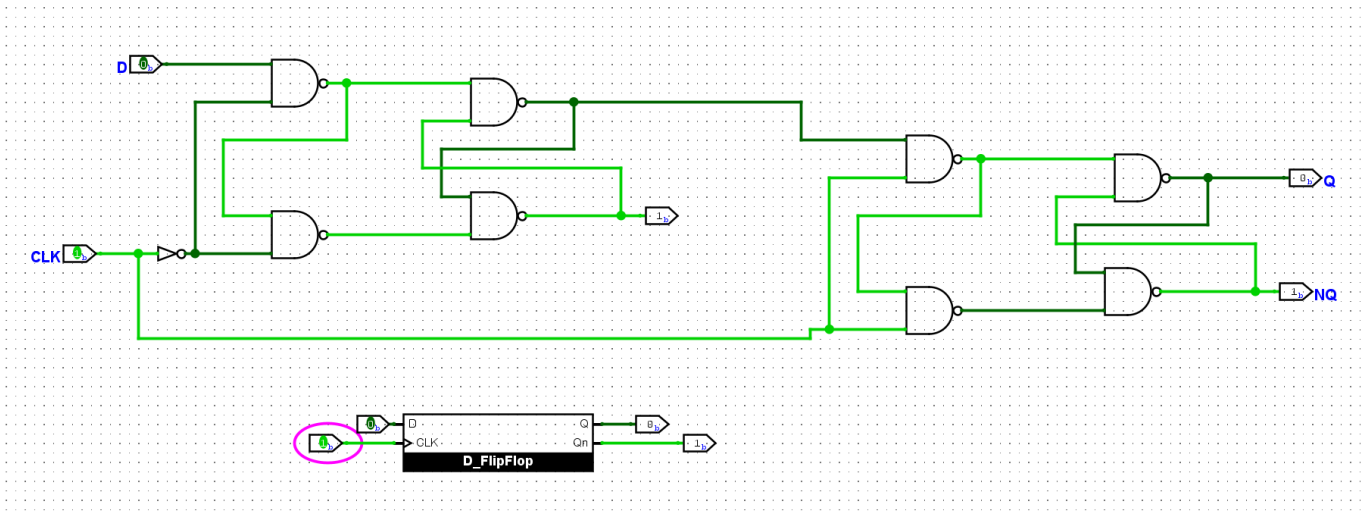


Figure 03.1. CLK=1, D=0: both implementations show Q=0, Qn=1 after storing zero.

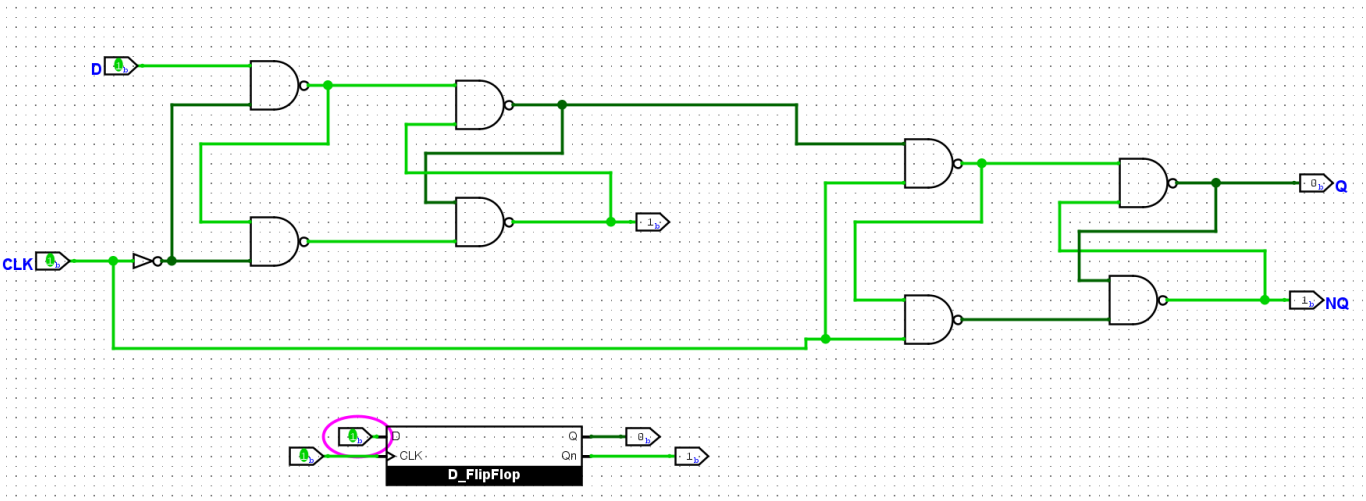


Figure 03.2. CLK remains 1 while D changes to 1: Q remains 0 and Qn remains 1 in both implementations. A D-only change does not update the output.

Positive-edge D Flip-Flop | Screenshots

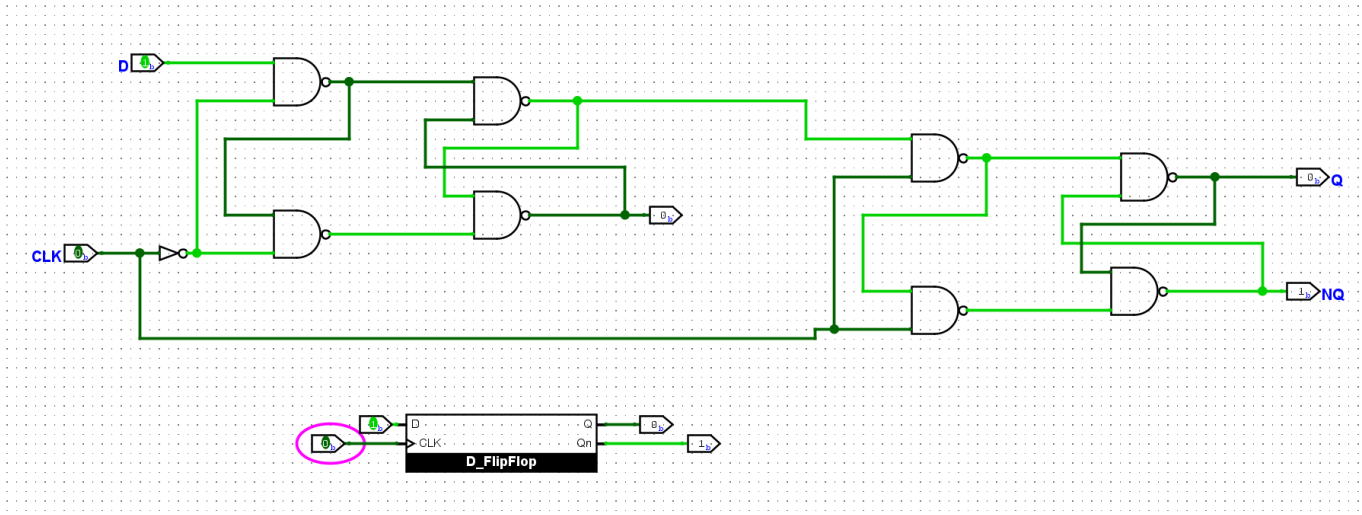


Figure 03.3. CLK=0, D=1: the master accepts 1 while the slave retains Q=0, Qn=1.

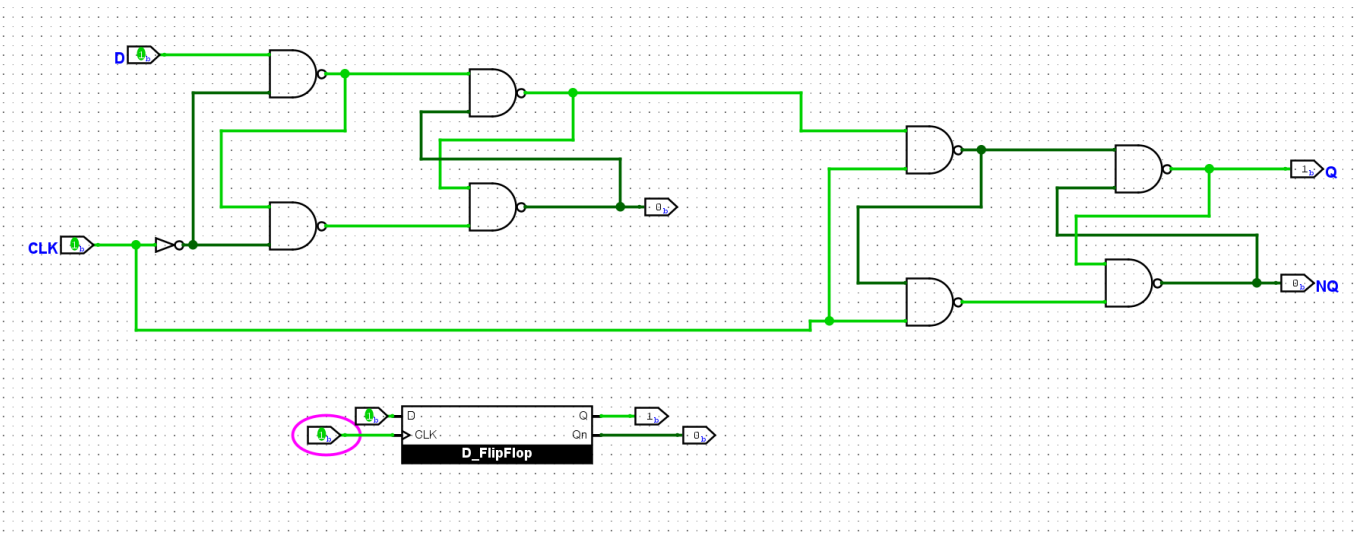


Figure 03.4. CLK changes from 0 to 1 with D=1: both implementations update to Q=1, Qn=0.

Positive-edge D Flip-Flop | Screenshots

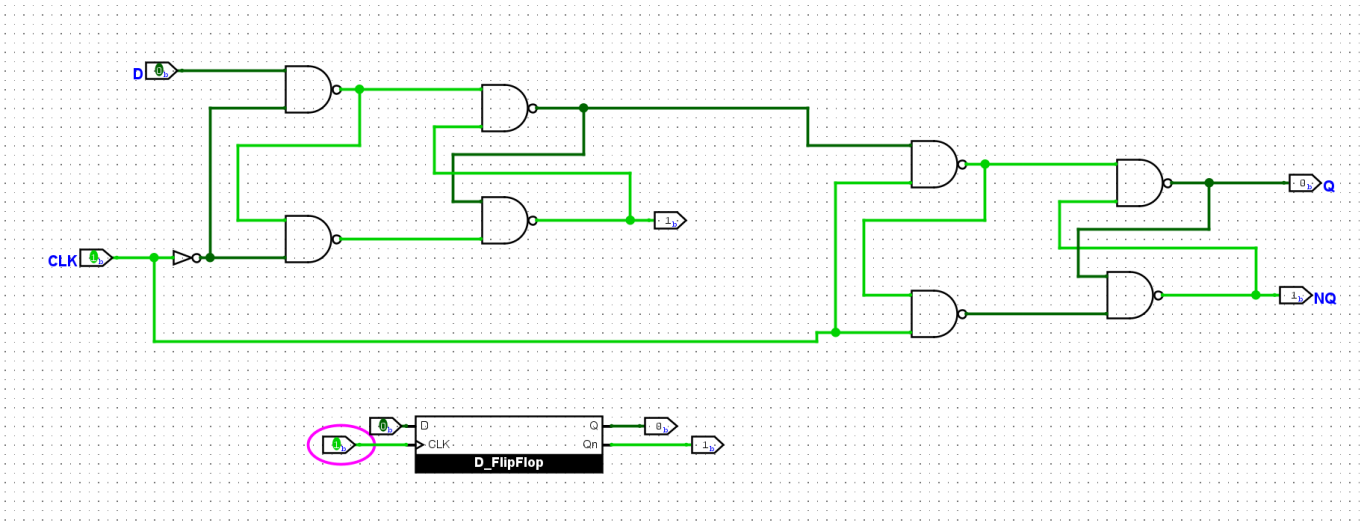


Figure 03.5. CLK=1, D=0: both implementations show Q=0, Qn=1 after the next capture. CLK must first return to 0 and rise again to store this zero.

JK Flip-Flop

Theory. A JK flip-flop generalizes the RS behavior by using feedback: at the active clock edge, J=K=0 holds, J=0/K=1 resets, J=1/K=0 sets, and J=K=1 toggles. Its ideal characteristic equation is Q(next) = J NOT Q(previous) OR (NOT K) Q(previous). A properly clocked master-slave design avoids continuous race-around in the ideal model.

J	K	Q(next) at rising edge	Function
0	0	Q(previous)	Hold
0	1	0	Reset
1	0	1	Set
1	1	NOT Q(previous)	Toggle

Verification procedure. Initialize the gate circuit with its auxiliary CLR, then release CLR to 0. Initialize the VHDL block separately using J=0, K=1 and a rising edge. Test Hold, Set and Reset. For Toggle, hold J=K=1 and keep CLR=0 while applying CLK=0, 1, 0, 1. Compare both outputs at each step: Q should be 0, 1, 1, 0 from a starting state of zero.

Result and scope. Use of the auxiliary CLR in the gate circuit was approved for this assignment. The VHDL block has J, K and CLK inputs; normal operation is compared with CLR=0 and matching initial states. Figures 04.7-04.10 record consecutive clock states 0, 1, 0, 1 with J=K=1. Both implementations produce Q=0, 1, 1, 0 and complementary Qn. Q toggles on each rising edge and holds on the falling edge. Figure 04.11 confirms this sequence for the gate-level circuit on a timing trace and shows initialization by CLR while CLK is low. The VHDL evidence is the consecutive captures in Figures 04.7-04.10. No equivalence is claimed for CLR=1.

VHDL Source | JK_FlipFlop

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity JK_FlipFlop is
    Port (
        J   : in  STD_LOGIC;
        K   : in  STD_LOGIC;
        CLK : in  STD_LOGIC;
        Q   : out STD_LOGIC;
        Qn  : out STD_LOGIC
    );
end JK_FlipFlop;

architecture Behavioral of JK_FlipFlop is
    signal q_state : STD_LOGIC;
begin

    process(CLK)
    begin
        if rising_edge(CLK) then

            if J = '0' and K = '0' then
                null;                   -- Hold

            elsif J = '0' and K = '1' then
                q_state <= '0';         -- Reset

            elsif J = '1' and K = '0' then
                q_state <= '1';         -- Set

            elsif J = '1' and K = '1' then
                q_state <= not q_state; -- Toggle

            end if;

        end if;
    end process;

    Q  <= q_state;
    Qn <= not q_state;

end Behavioral;
```

The rising-edge process implements Hold, Reset, Set and Toggle. There is no CLR port. Reset or Set must establish a known state before a meaningful Toggle test.

JK Flip-Flop | Circuit Diagram

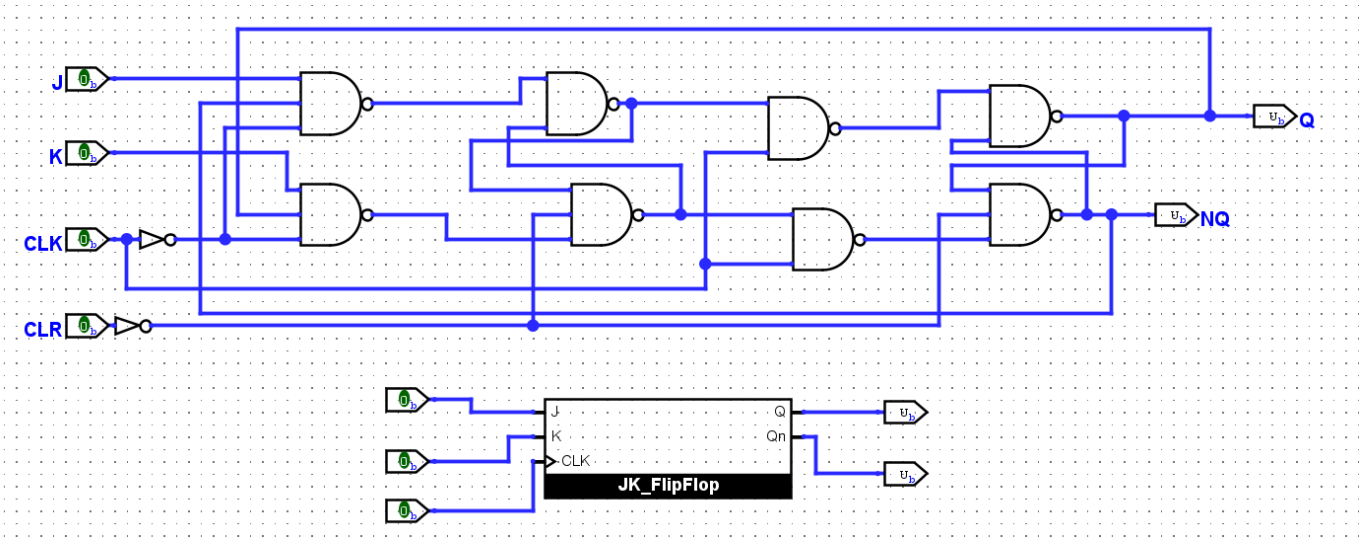


Figure 04.D. Circuit diagram. JK Flip-Flop.

Why CLR Was Added to the JK Gate Circuit

Purpose: establish a known starting state. Permission was obtained to use CLR in the JK gate circuit for this assignment. The auxiliary CLR provides a direct way to initialize the cross-coupled gate circuit before testing its JK behavior. A feedback storage circuit does not inherently guarantee a particular initial Q value. Without a defined starting state, a hold or toggle result can be difficult to interpret because it depends on the state already stored.

How it supports the experiment. CLR is intended to force the gate circuit into the reset state (Q=0, Qn=1) independently of the normal J/K clocked operation. After initialization, CLR is returned to 0 so that hold, set, reset and toggle can be examined from a known state. This makes the test setup repeatable and separates initialization from the operation being tested.

Relation to the VHDL implementation. The VHDL component has J, K and CLK inputs but no CLR input. Its starting state must therefore be established separately, for example by applying J=0, K=1 and a rising clock edge. Comparison of normal JK operation requires CLR=0 on the gate circuit, the same initial Q, and matching J/K and clock actions. The two interfaces are different, so complete equivalence is not claimed when CLR=1.

Test observations. CLR remains 0 during the JK functional tests. In the consecutive Toggle sequence, both implementations produce Q=0, 1, 1, 0 for CLK=0, 1, 0, 1 while J=K=1. The stored value reverses at each rising edge and remains unchanged at the falling edge. Figure 04.11 also shows the CLR initialization pulse before normal Toggle operation.

JK Flip-Flop | Screenshots

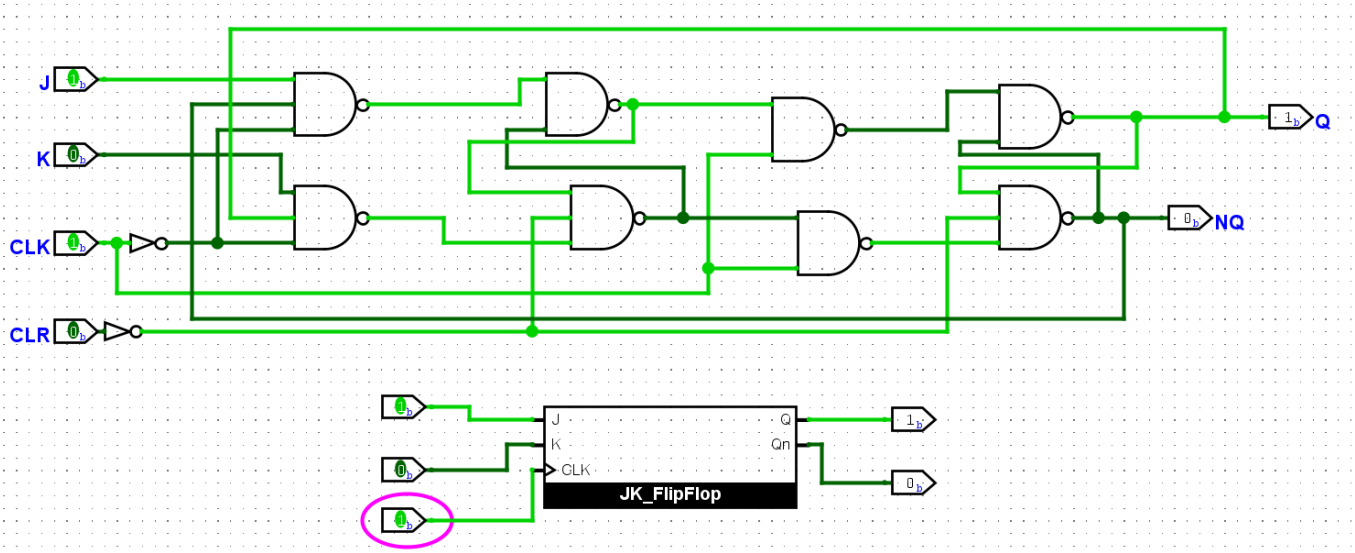


Figure 04.1. J=1, K=0, CLK=1, CLR=0: both show Q=1, Qn=0. Set configuration, consistent with a preceding rising edge.

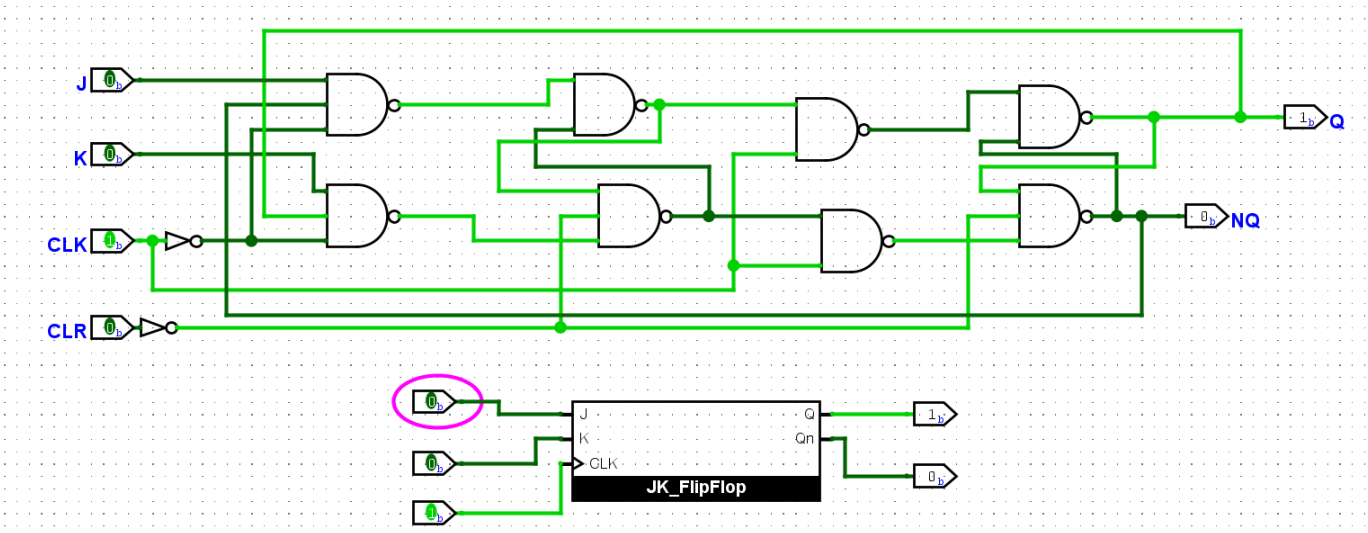


Figure 04.2. J=0, K=0, CLK=1, CLR=0: both again show Q=1.

JK Flip-Flop | Screenshots

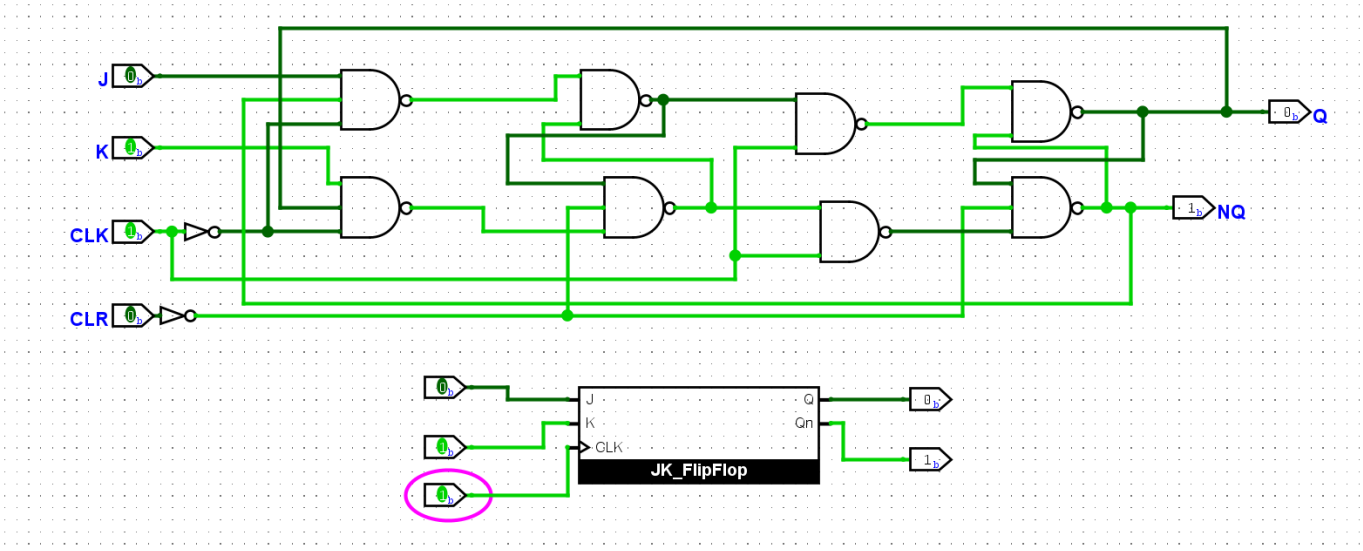


Figure 04.3. J=0, K=1, CLK=1, CLR=0: both show Q=0. Consistent with clocked reset.

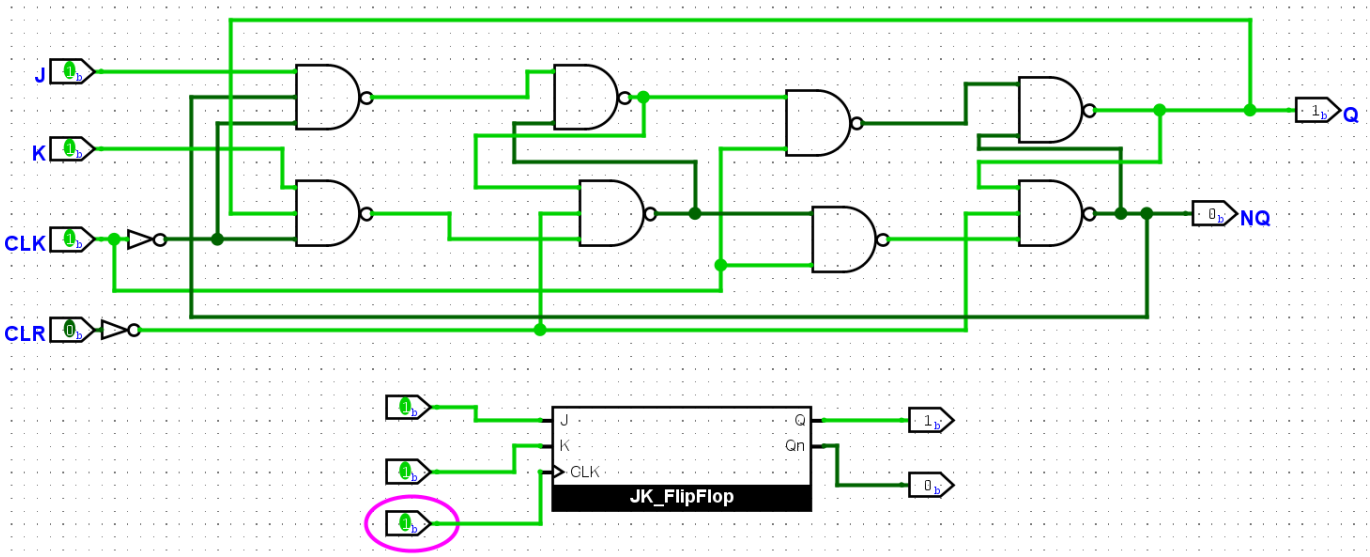


Figure 04.4. J=1, K=1, CLK=1, CLR=0: both show Q=1. Toggle-state configuration. The consecutive Toggle test is shown in Figures 04.7-04.10.

JK Flip-Flop | Screenshots

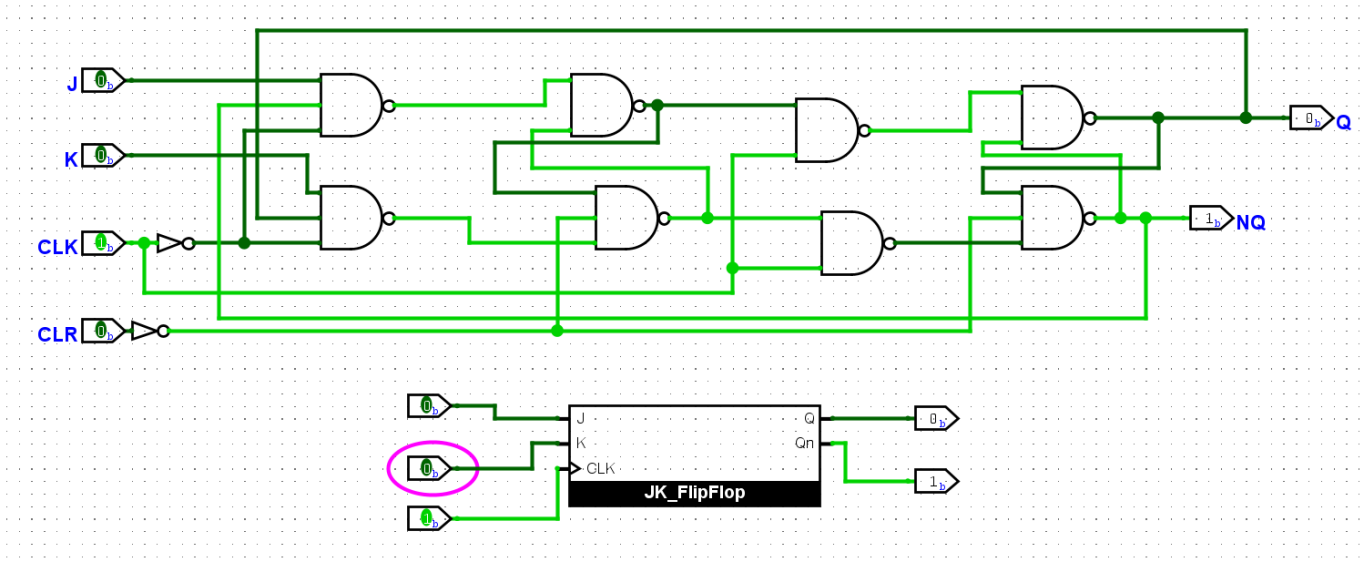


Figure 04.5. J=0, K=0, CLK=1, CLR=0: both show Q=0. Independent hold-zero configuration.

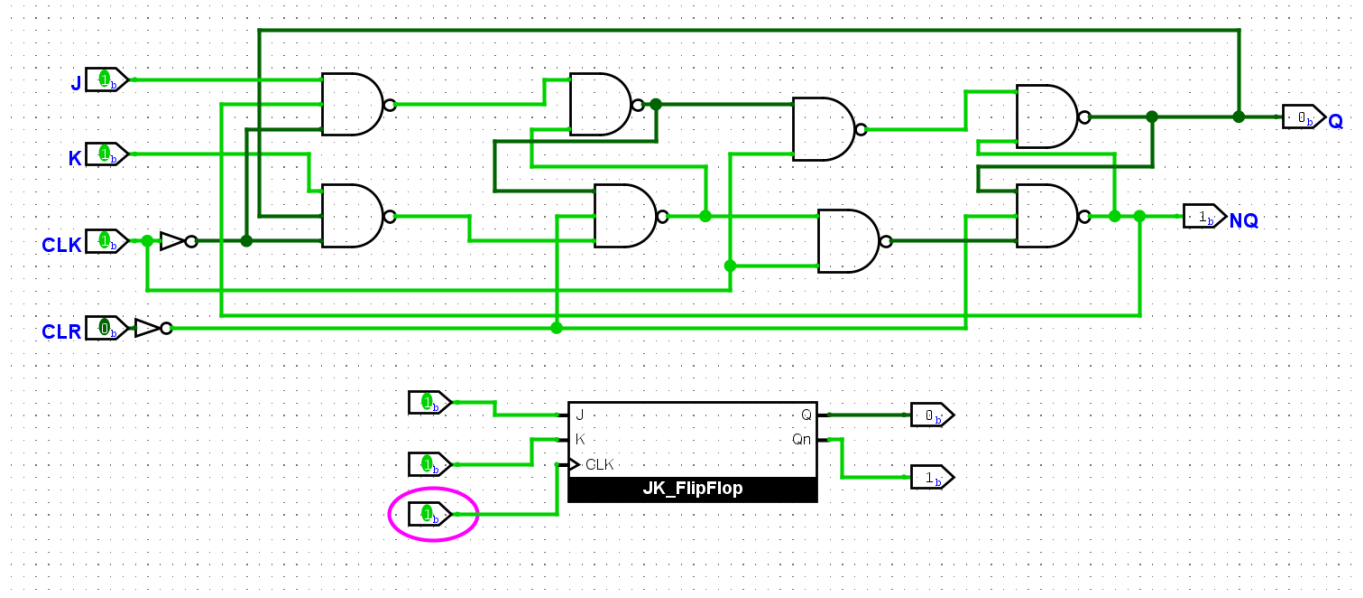


Figure 04.6. J=1, K=1, CLK=1, CLR=0: both show Q=0. Toggle-state configuration. Figures 04.7-04.10 document the clock sequence and both output transitions.

JK Flip-Flop | Sequential Toggle Test

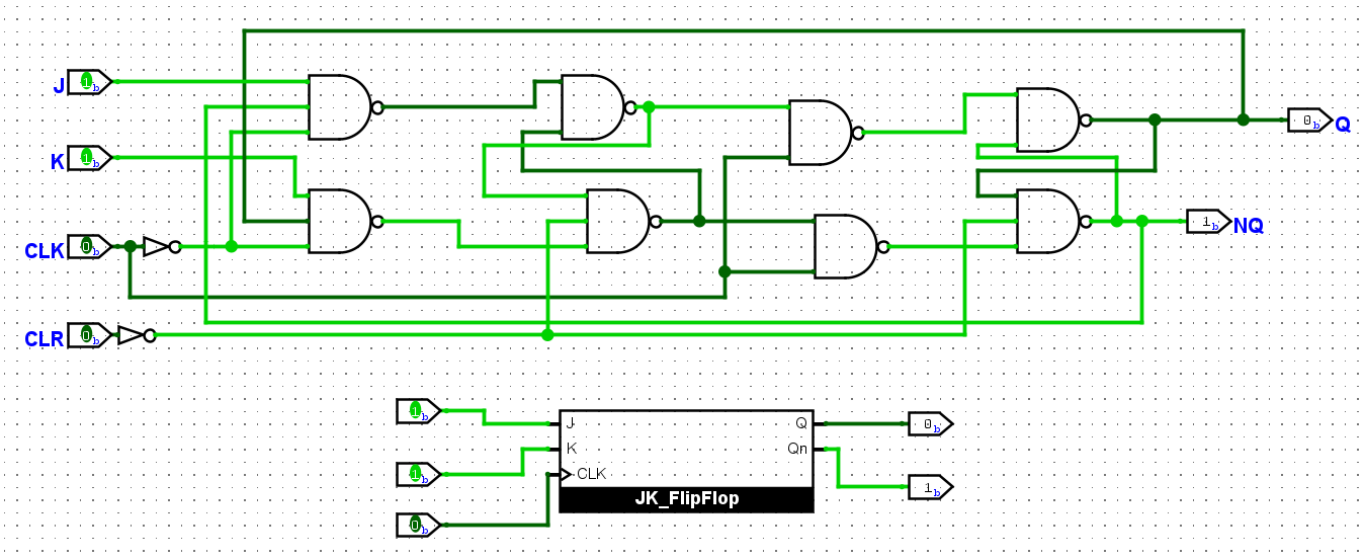


Figure 04.7. Toggle step 1: J=K=1, CLR=0, CLK=0. Both implementations start with Q=0, Qn=1.

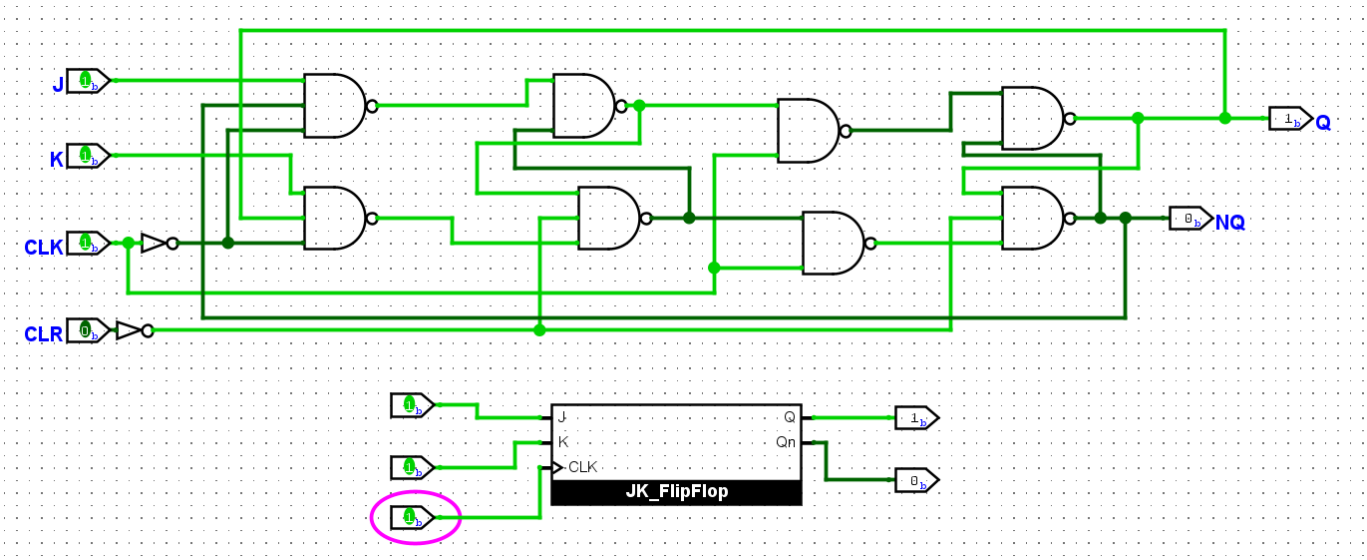


Figure 04.8. Toggle step 2: CLK rises from 0 to 1 with J=K=1 and CLR=0. Both outputs toggle to Q=1, Qn=0.

JK Flip-Flop | Sequential Toggle Test

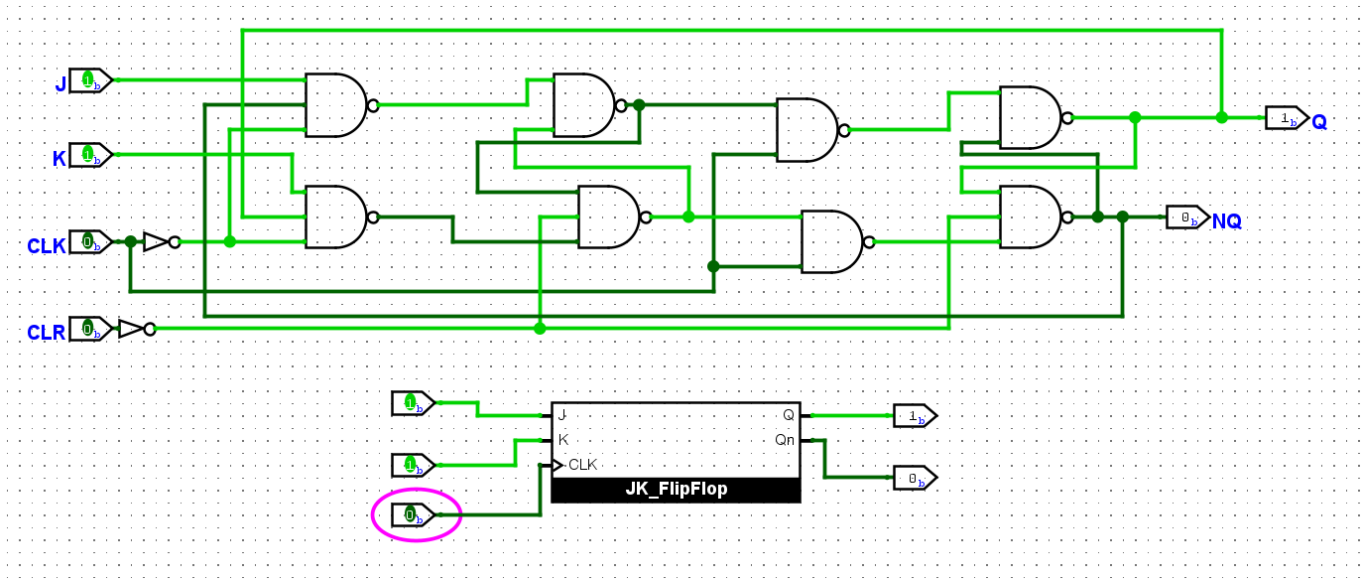


Figure 04.9. Toggle step 3: CLK falls from 1 to 0. With J=K=1 and CLR=0, both outputs hold Q=1, Qn=0.

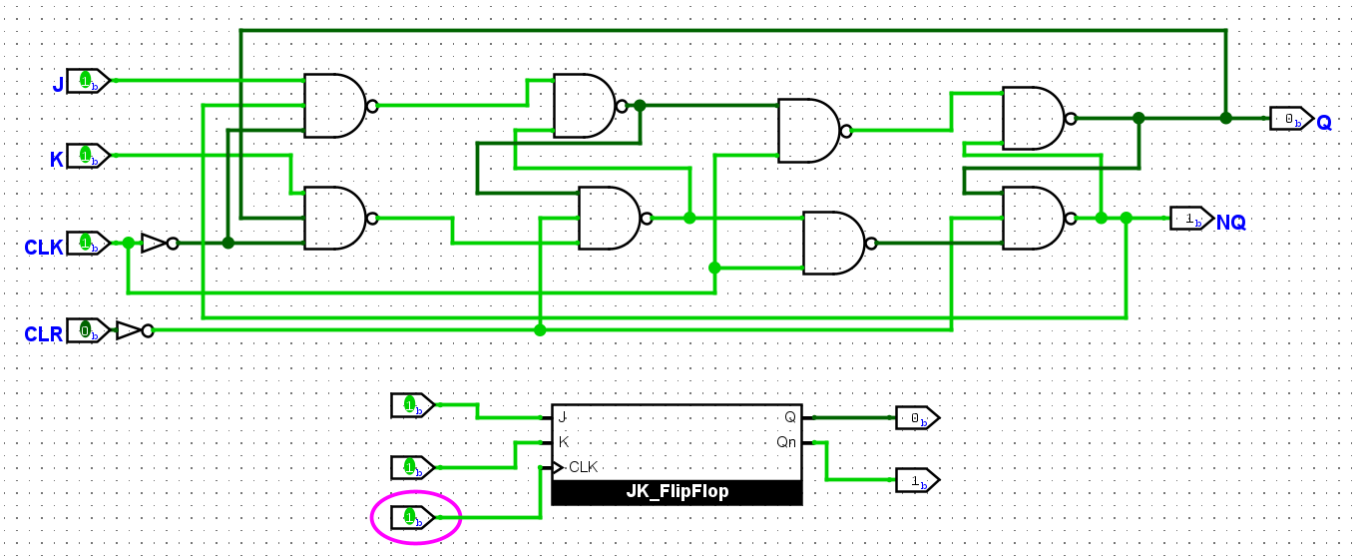


Figure 04.10. Toggle step 4: CLK rises again from 0 to 1. Both outputs toggle back to Q=0, Qn=1, completing the 0-to-1-to-0 sequence.

JK Flip-Flop | Timing Diagram

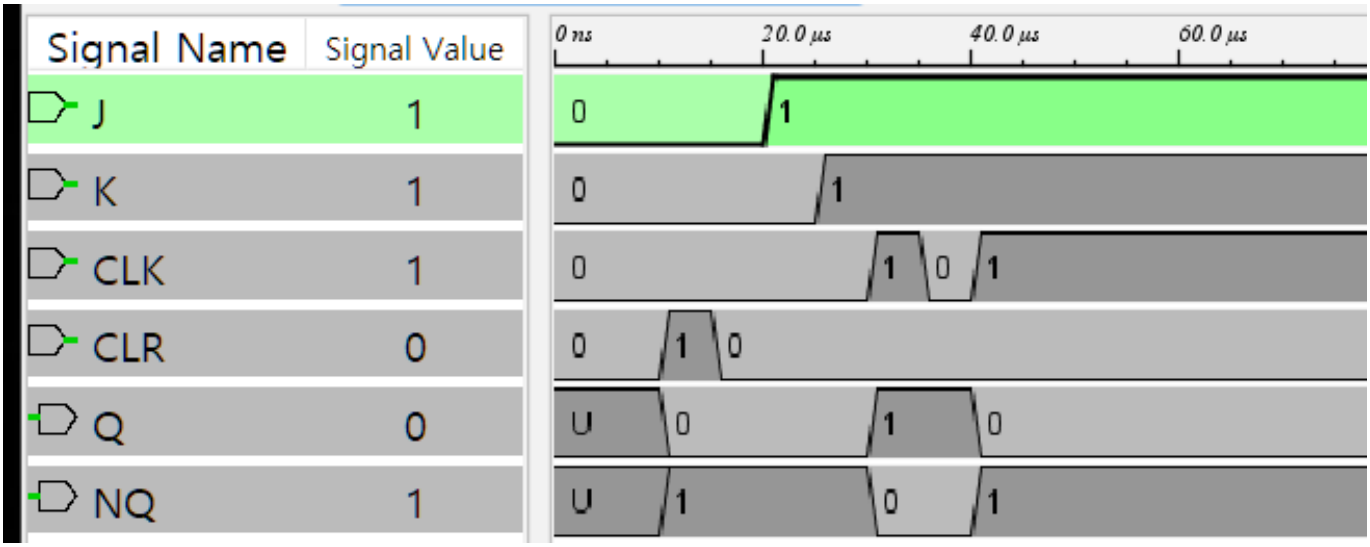


Figure 04.11. Logisim-evolution timing diagram of the gate-level JK circuit with auxiliary CLR. NQ is the complementary output Qn.

Event	Input condition	Observed Q / NQ
Initialization	CLR rises while CLK=0	U / U becomes 0 / 1
First rising edge	J=K=1; CLR=0	0 / 1 becomes 1 / 0
Falling edge	J=K=1; CLR=0	1 / 0 is retained
Second rising edge	J=K=1; CLR=0	1 / 0 becomes 0 / 1

Initialization. At the beginning, Q and NQ are undefined (U). The CLR pulse establishes Q=0 and NQ=1 while CLK remains low. After CLR returns to 0, J and K are both set to 1 before the first rising clock edge.

Toggle verification. The first rising edge changes Q from 0 to 1. Q remains 1 through the falling edge and changes back to 0 at the next rising edge. NQ follows the complementary sequence. The trace directly supports positive-edge Toggle operation and agrees with the consecutive circuit captures in Figures 04.7-04.10.

Measurement scope. This trace records only the gate-level JK circuit, including its auxiliary CLR; it does not record the VHDL component outputs. The displayed time axis represents simulation time; no physical propagation-delay measurement is inferred.

D Flip-Flop with Asynchronous Set/Clear

Theory. Active-high asynchronous controls override clocked storage. Clear has first priority, Set has second priority, and D is captured on a rising edge only when both controls are low. The VHDL implementation uses a CLK port and rising_edge(CLK), explicitly identifying the clock.

CLR	SET	Clock event	Q(next)	Function
1	X	X	0	Clear priority
0	1	X	1	Async Set
0	0	Rising edge	D	Capture
0	0	No rising edge	Q(previous)	Hold

Verification procedure. Keep CLK low and assert CLR, then test SET with CLR low. Assert both controls to check Clear priority. Release both; capture D=1 with a rising edge. Change D to 0 while holding CLK high and check retention, then pulse CLK low-to-high to capture zero.

Result and scope. The VHDL implementation uses if clr, elsif set, elsif rising_edge(CLK), matching the priority table. Omitting D from this edge-triggered process sensitivity list is appropriate: a D-only event should not update state. The seven test images show the expected output states, including Q=0 when both controls are high. In Figures 05.1-05.3, CLK is 0 in every capture while Q takes the value selected by Clear or Set. These states are consistent with asynchronous control that does not require a clock edge; the static captures do not measure the response time. Hardware mapping of simultaneous asynchronous controls is device-dependent.

VHDL Source | D_FF_Async

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FF_Async is
    Port (
        D   : in  STD_LOGIC;
        CLK : in  STD_LOGIC;
        set : in  STD_LOGIC;
        clr : in  STD_LOGIC;
        Q   : out STD_LOGIC;
        Qn  : out STD_LOGIC
    );
end D_FF_Async;

architecture Behavioral of D_FF_Async is
    signal q_state : STD_LOGIC;
begin

    process(CLK, set, clr)
    begin
        if clr = '1' then
            q_state <= '0';

        elsif set = '1' then
            q_state <= '1';

        elsif rising_edge(CLK) then
            q_state <= D;

        end if;
    end process;

    Q  <= q_state;
    Qn <= not q_state;

end Behavioral;
```

The process checks clr first, set second and rising_edge(CLK) third, implementing Clear priority and asynchronous controls.

D Flip-Flop with Asynchronous Set/Clear | Circuit Diagram

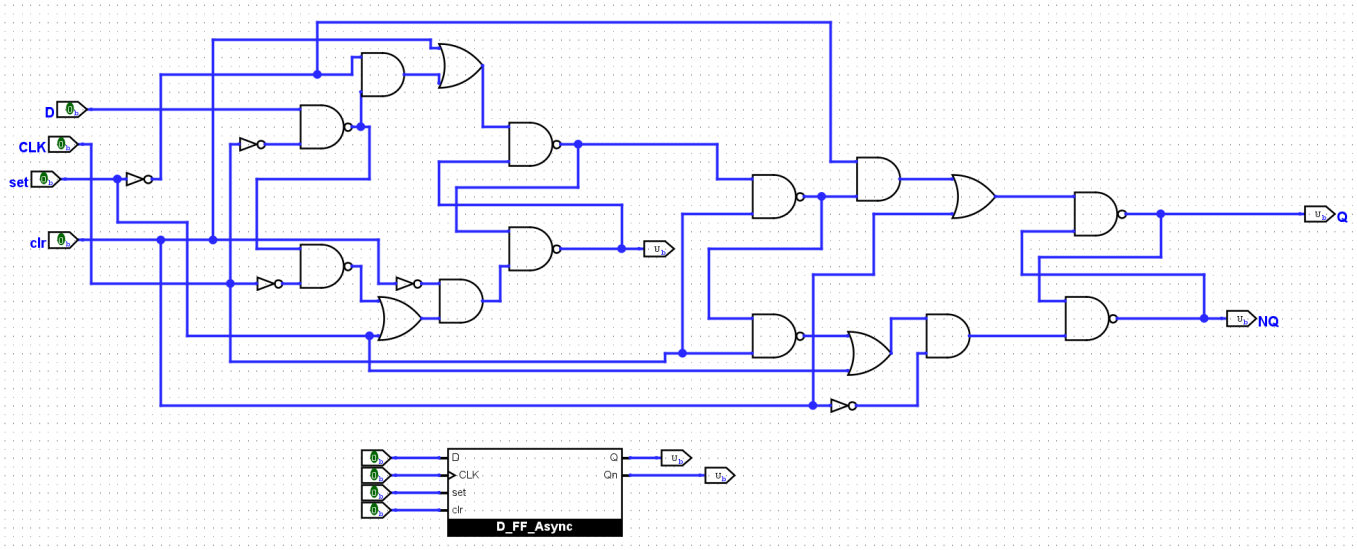


Figure 05.D. Circuit diagram. D Flip-Flop with Asynchronous Set/Clear.

D Flip-Flop with Asynchronous Set/Clear | Screenshots

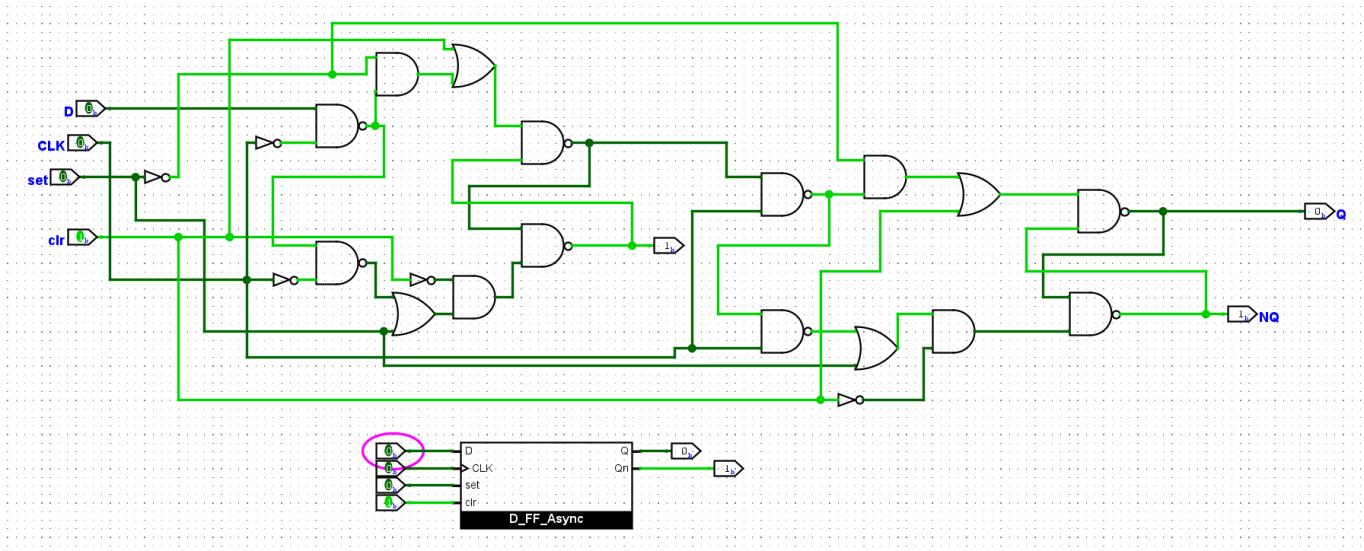


Figure 05.1. D=0, CLK=0, SET=0, CLR=1: both show Q=0, Qn=1. Clear asserted.

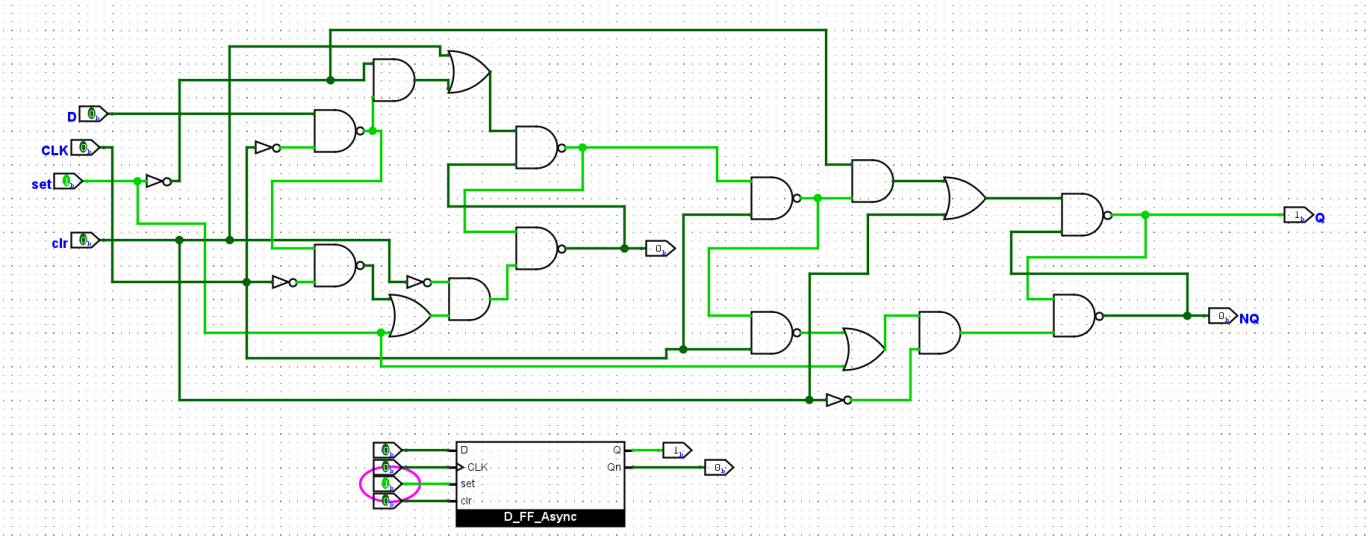


Figure 05.2. D=0, CLK=0, SET=1, CLR=0: both show Q=1, Qn=0. Set asserted.

D Flip-Flop with Asynchronous Set/Clear | Screenshots

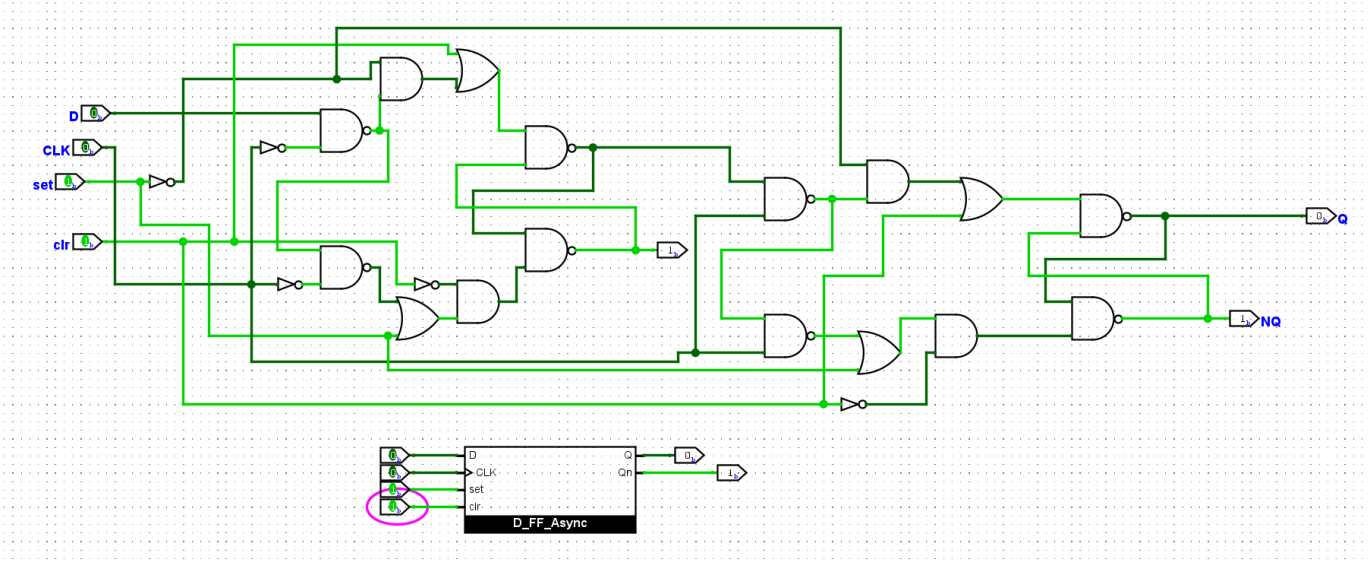


Figure 05.3. D=0, CLK=0, SET=1, CLR=1: both show Q=0, Qn=1. Clear priority.

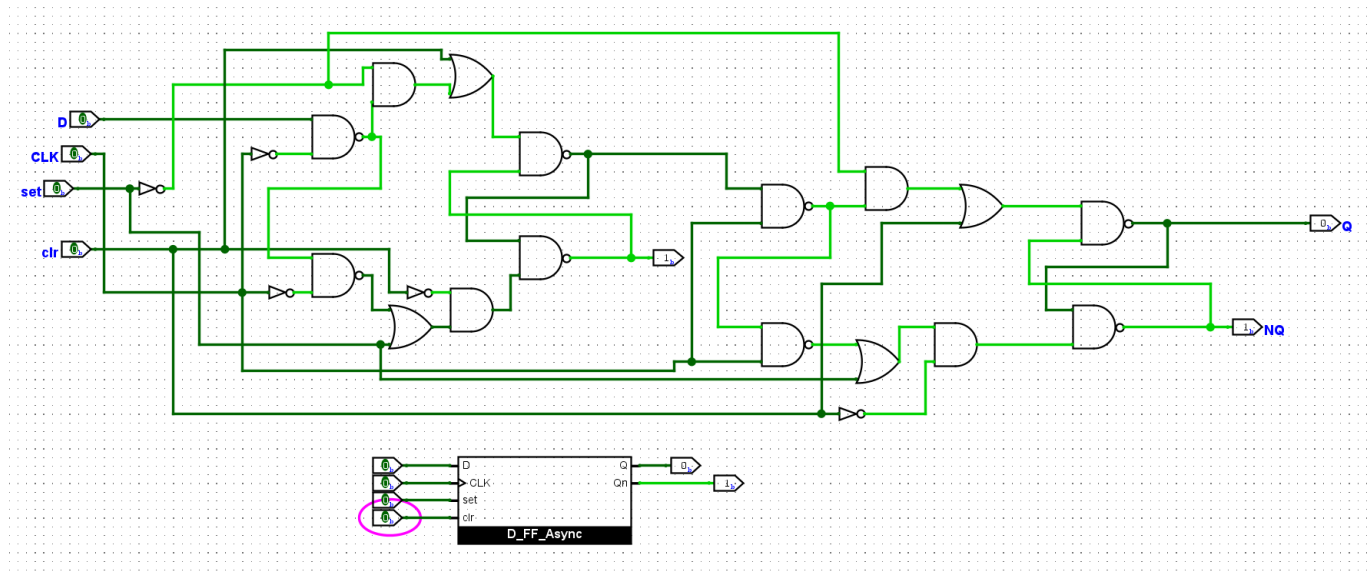


Figure 05.4. D=0, CLK=0, SET=0, CLR=0: Q=0, Qn=1. Consistent with retaining zero after releasing the controls.

D Flip-Flop with Asynchronous Set/Clear | Screenshots

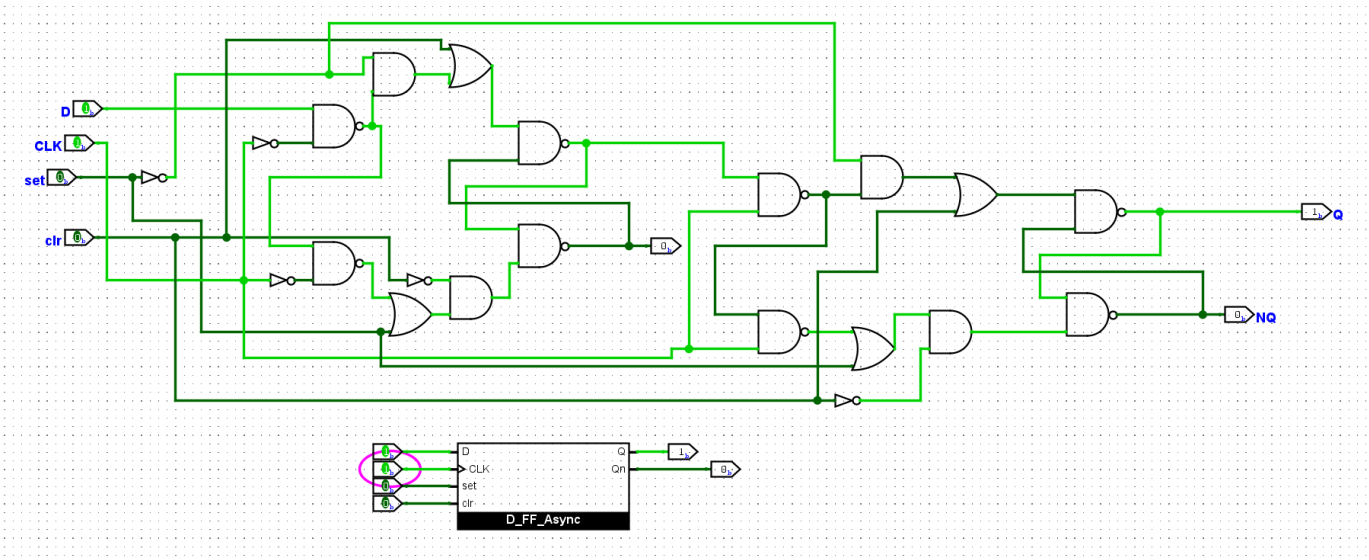


Figure 05.5. D=1, CLK=1, controls low: Q=1. Consistent with capture after a rising edge.

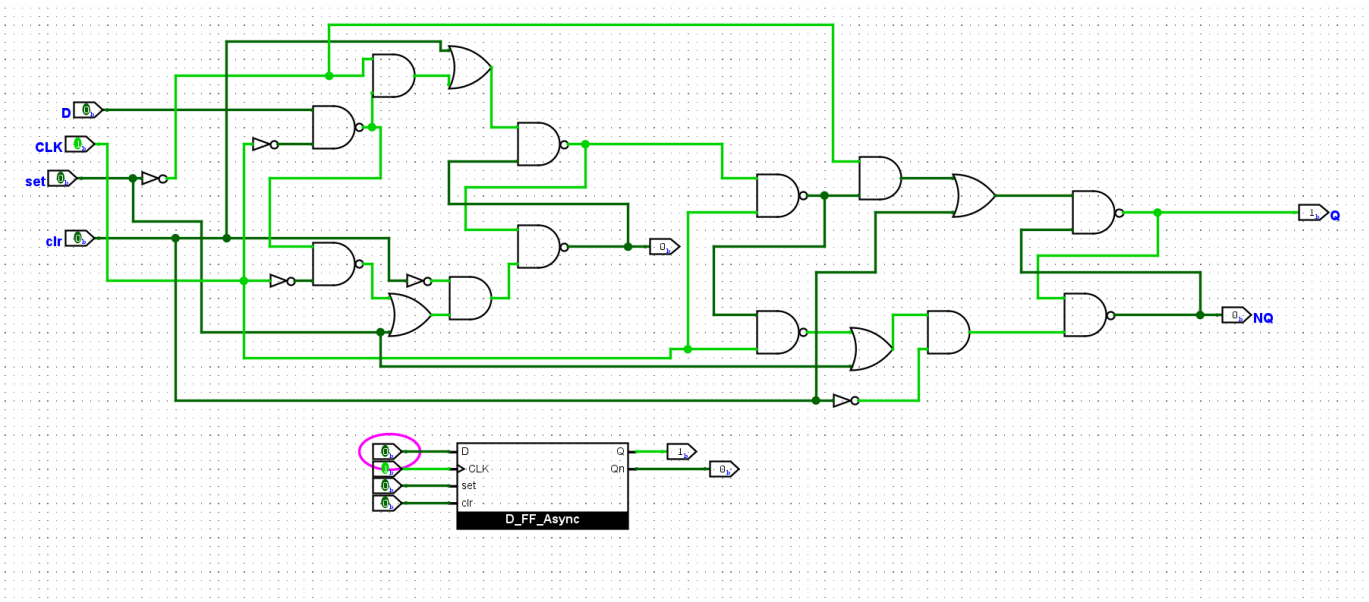


Figure 05.6. D=0, CLK=1, controls low: Q=1. Consistent with retention after changing D only.

D Flip-Flop with Asynchronous Set/Clear | Screenshots

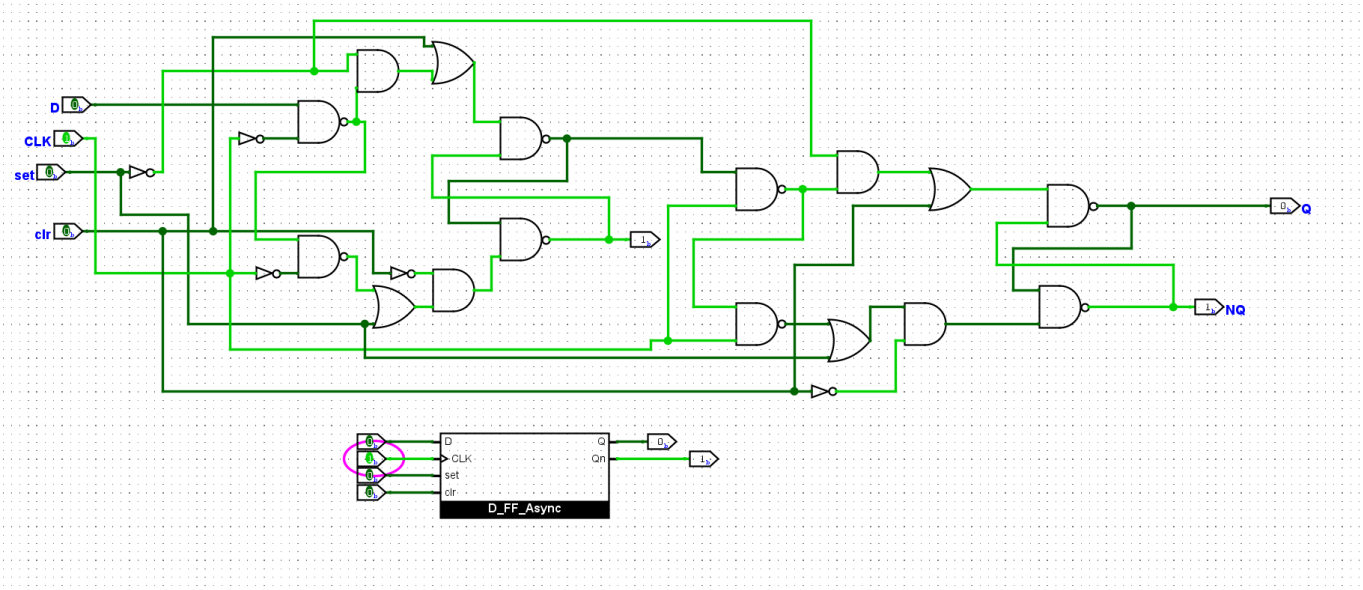


Figure 05.7. D=0, CLK=1, controls low: Q=0. A new rising edge is needed to capture zero.

Findings and Limitations

Experimental environment. Logisim-evolution was used for gate-level circuit construction and circuit simulation. The VHDL source code and corresponding component outputs are included alongside the gate-level tests. The JK timing diagram records initialization and clocked output transitions of the gate-level circuit in the same simulation environment. The external VHDL simulator and its version have not been identified, so no specific compilation backend is attributed to these results.

The screenshots illustrate feedback memory, level-sensitive transparency, clocked data storage, JK state selection and asynchronous priority. Within each displayed configuration, the gate circuit and VHDL block show matching outputs. That agreement is narrower than a proof that the implementations behave identically for every input history.

JK verification. Figures 04.1-04.6 demonstrate Set, Hold, Reset and toggle-state configurations. Figures 04.7-04.10 provide the consecutive Toggle test: with J=K=1 and CLR=0, CLK follows 0, 1, 0, 1 and Q follows 0, 1, 1, 0 in both implementations. Figure 04.11 confirms the same behavior of the gate-level circuit on a timing trace: Q rises on the first rising clock edge, holds through the falling edge, and falls on the second rising edge. The initial CLR pulse establishes Q=0 while CLK is low. VHDL Toggle behavior is documented by Figures 04.7-04.10, not by this waveform. CLR is an approved initialization aid.

Timing and implementation limits. Static pictures cannot establish propagation delay, setup/hold compliance, recovery/removal timing, or metastability behavior. The RS forbidden-state release is indeterminate in physical hardware. Initial state is not assumed unless established by an input sequence.

Implementation workflow. The experiment used Logisim-evolution only; no separate synthesis flow was performed. The circuit diagrams in this report are manually constructed gate-level schematics, rather than schematics generated by a synthesis tool. The JK timing trace verifies clocked Toggle behavior of the gate-level circuit. For Exercise 5, Figures 05.1-05.3 show CLK=0 with Q controlled by Set/Clear, consistent with operation that does not require a clock edge. The remaining images illustrate normal D flip-flop operation.